



Deep Learning Approaches for Modelling Economic Time Series and Financial Returns

A Comparative Study with Econometric Time Series Models

Master Thesis

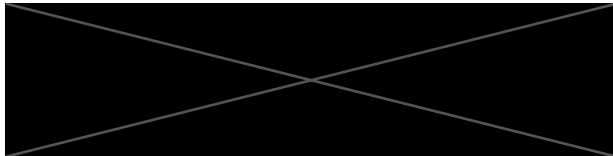
In the Department for Econometrics and Quantitative Methods

Submitted by: Leon Behrens

Supervised by: Prof. Yuanhua Feng

Submitted on: September 29, 2025

Study programme: Management Information Systems



Paderborn, 29. September 2025

Abstract

This thesis conducts a comparative study of econometric time series models (ARIMA, ARCH/GARCH-type models) and deep learning architectures (FFNN, RNN, LSTM) in forecasting one-step ahead returns and volatility using historical S&P 500 data. It outlines the intricacies of financial time series analysis and forecasting tasks, providing insights into the motivation behind econometric models. By outlining not only their strengths but their weaknesses, this thesis motivates exploring the application of deep learning approaches for modelling economic time series. This comparative study aims to strengthen its robustness by using a 12-fold cross validation approach and exploring alternative volatility proxies. While eleven models are compared across two forecasting tasks, the results suggest that increased complexity does not necessarily correlate with increased accuracy.

Acknowledgements

To my supervisor, Yuanhua Feng, Professor in Econometrics and Quantitative Methods at Paderborn University, I would like to express my deepest gratitude for both the opportunity and the support throughout this process.

To my friends and family, I appreciate you for your support and for refraining from continuously asking me how it is going and how many pages I have already written. To those of you who took their time to proof-read this and especially the one who proof-read countless term papers over the years, thank you!

To Simbiatu Abisola Abiola Ajikawo, thank you from the bottom of my heart for keeping me focused, motivated, and most importantly, sane. And finally, to Sampha Lahai Sisa, thank you for letting this all work out.

Table of Contents

List of Figures	II
List of Tables	III
List of Abbreviations	IV
1 Introduction	1
2 Literature Review and Theoretical Background	4
2.1 Financial Time Series Data	4
2.2 Econometric Time Series Models	8
2.3 Deep Learning Approaches	10
2.4 Hybrid Approaches	12
2.5 Model Performance Evaluation	13
2.6 Hypotheses	16
3 Models.....	19
3.1 Econometric Time Series Models	19
3.1.1 Autoregressive Integrated Moving Average (ARIMA)	19
3.1.2 Autoregressive Conditional Heteroscedasticity (ARCH)	21
3.1.3 Generalised Autoregressive Conditional Heteroscedasticity (GARCH)	22
3.1.4 GARCH-Extensions.....	24
3.2 Deep Learning Models.....	24
3.2.1 Feed-Forward Neural Networks (FFNN).....	24
3.2.2 Recurrent Neural Networks (RNN)	27
3.2.3 Long Short-Term Memory (LSTM).....	29
4 Methodology.....	32
4.1 Data Collection & Description.....	32
4.2 Data Cleaning and Preparation	33
4.3 Exploratory Data Analysis (EDA)	34
4.3.1 EDA Return Dataset.....	34
4.3.2 EDA Volatility Dataset.....	36
4.4 Model Evaluation Metrics.....	38
4.5 Time Series Cross-Validation.....	38
4.6 Model Implementation	40
4.6.1 ARIMA	40
4.6.2 ARCH, GARCH, and GARCH-Extensions.....	42
4.6.3 DL Models	43
4.6.4 Hybrid Models	46
5 Results.....	48
5.1 Forecasting Returns	48
5.2 Forecasting Volatility	50
5.3 Extension: Forecasting Volatility	54
6 Discussion	57
6.1 Model Performances when Forecasting Returns	57

Leon Behrens	II
DL Approaches for Modelling Economic Time Series and Financial Returns	
6.2 Model Performances when Forecasting Volatility	58
6.3 Limitations	61
7 Conclusion	64
Reference List	V
Statutory declaration	X
Appendix	XI

List of Figures

Figure 1: Daily Closing Prices (L) and Returns (R) of Apple from Jan 2021 to Mar 2025.	5
Figure 2: Correlogram of Returns (L) and Squared Returns (R) of Apple in Q1 2021.	6
Figure 3: Returns of Microsoft from January 1990 to March 2025.....	8
Figure 4: FFNN with Three Hidden Layers following James et al., (2021, p. 409).	25
Figure 5: Schematic of a Simple RNN following James et al. (2021, p. 422).	28
Figure 6: LSTM Structure following Olah (2015).	30
Figure 7: S&P 500 Closing Price (Jan 2015 to Dec 2024).	35
Figure 8: Return Series of S&P 500 (Jan 2015 to Dec 2024).	36
Figure 9: S&P 500 Closing Price (Jan 1995 to Dec 2024).	36
Figure 10: Return Series of S&P 500 (Jan 1995 to Dec 2024).	37
Figure 11: Train/Test Split Return Dataset.....	38
Figure 12: Train/Test Split Volatility Dataset.....	39
Figure 13: Lengths Training/Test Data (Return Dataset).	39
Figure 14: Lengths Training/Test Data (Volatility Dataset).	40
Figure 15: General Process ARIMA following Hyndman & Athanasopoulos (2014, p. 235).	42
Figure 16: PACF to Determine ARCH Order.	43

List of Tables

Table 1: Common Volatility Estimators.....	16
Table 2: Variable Descriptions.	32
Table 3: Structure of the Dataset.....	32
Table 4: Structure of the Final Return Dataset.....	34
Table 5: Structure of the Final Volatility Dataset.....	34
Table 6: ADF-statistics of Return Series in Return and Volatility Dataset.	34
Table 7: Descriptive Statistics of the Returns (Jan 2015 to Dec 2024)	35
Table 8: Descriptive Statistics of the Returns (Jan 1995 to Dec 2024).	37
Table 9: Descriptive Statistics of the Daily Volatility Estimates (Jan 1995 to Dec 2024).....	38
Table 10: ARIMA Order and Ljung-Box Test Results.....	41
Table 11: Model Validation (Return Forecasts).	44
Table 12: Model Validation (Volatility Forecasts).	44
Table 13: DL Hyperparameters (Return Forecasts).	45
Table 14: DL Hyperparameters (Volatility Forecasts).	45
Table 15: Ljung-Box Test Results for ARCH/GARCH-type models.	47
Table 16: Model Evaluation (Return Forecasts).	48
Table 17: MSE (e-05) Across All Twelve Folds (Return Forecasts).....	49
Table 18: MAE (e-03) Across All Twelve Folds (Return Forecasts).	50
Table 19: Model Evaluation (Volatility Forecasts).	51
Table 20: MSE (e-09) Across All Twelve Folds (Volatility Forecasts).....	52
Table 21: QLIKE Across All Twelve Folds (Volatility Forecasts).....	53
Table 22: Model Evaluation (Volatility Forecasts with Proxy).....	55
Table 23: MSE (e-09) Across All Twelve Folds (Volatility Forecasts with Proxy).	55
Table 24: QLIKE Across All Twelve Folds (Volatility Forecasts with Proxy).	56

List of Abbreviations

ACF	Autocorrelation function
AIC	Akaike Information Criterion
ARCH	Autoregressive conditional heteroscedasticity
ARMA	Autoregressive moving average
ARIMA	Autoregressive integrated moving average
BPTT	backpropagation through time approach
DL	Deep learning
EDA	Exploratory data analysis
e.g.	For example
eq.	Equation
et al.	And others
FFNN	Feed-forward neural network
GARCH	Generalised autoregressive conditional heteroscedasticity
GDP	Gross domestic product
iid	Independent and identically distributed
LSTM	Long short-term memory
MAE	Mean absolute error
MAPE	Mean absolute percent error
ME	Mean error
ML	Machine learning
MLP	Multilayer perceptron
MSE	Mean square error
NN	Neural network
ReLU	Rectified linear unit
RMSE	Root mean square error
RNN	Recurrent neural network
SD	Standard deviation
S&P 500	Standard & Poor's 500

1 Introduction

Applying statistical theory and methods constitutes an important part of analysing financial time series data (Tsay, 2010). Modelling and accurately forecasting metrics such as returns and volatility of financial assets can provide valuable information to support investment decision-making (Wu et al., 1996; Heaton et al., 2017), with forecasting volatility, for example, playing a key role in financial risk management (Bollerslev et al., 1992). Carefully defining the underlying problem, as well as gathering and understanding the relevant data are indispensable steps when approaching forecasting tasks (Hyndman & Athanasopoulos, 2014). Crucially, financial time series data distinguishes itself from other time series data, as it is characterised by, for example, heteroscedasticity, leverage effects, and seasonality (Tsay, 2010). Moreover, unlike many time series in natural science, financial time series tend to display non-stationarity (Francq & Zakoian, 2010; Wu et al., 1996). Hence, these characteristics must be considered when modelling and forecasting in economics and finance.

As a response, numerous econometric time series models have been developed and refined over time. Notable models include, for example, *autoregressive integrated moving average* (ARIMA) models to deal with non-stationary time series (Box et al., 1994), as well as *autoregressive conditional heteroscedasticity* (ARCH) models (Engle, 1982) and *generalised ARCH* (GARCH) models to account for the heteroscedasticity of financial time series (Bollerslev, 1986). Nonetheless, appropriate model selection based on the underlying forecasting problem and the available data is crucial and commonly involves comparing two or three potential models (Hyndman & Athanasopoulos, 2014). For instance, while ARIMA models were designed to capture the mean dynamics of return series (Box et al., 1994), ARCH or GARCH-type models, on the other hand, excel at capturing the conditional variance of returns when modelling volatility (Bollerslev, 1986; Engle, 1982; Francq & Zakoian, 2010; Tsay, 2010).

However, in addition to comparing just econometric time series models, recent advances in the fields of *machine learning* (ML) and *deep learning* (DL) (LeCun et al., 2015), saw DL architectures emerge as serious competitors in finance (Heaton et al., 2017), as their ability to learn complex, non-linear patterns directly from the data makes them well-suited for financial forecasting tasks (Zhang et al., 1998). But while DL provides a powerful way to make quality predictions by their ability to learn patterns directly from

the data, this very ability also constitutes a key weakness, as they make mostly unverifiable assumptions (Mullainathan & Spiess, 2017), highlighting the trade-off between prediction accuracy and model interpretability (James et al., 2021). Moreover, previous research shows promising albeit mixed results that highlight the benefits and drawbacks of using DL models in forecasting tasks (Henrique et al., 2019; Makridakis et al., 2018; Sezer et al., 2020).

Hence, this thesis uses historical daily data from the Standard & Poor's 500 (S&P 500) to compare the forecasting performances of econometric time series models and DL architectures. Econometric time series models' forecasting performance is represented by ARIMA for one-step ahead log-return forecasts and ARCH/GARCH-type models for one-step ahead volatility forecasts and is later compared against *feed-forward neural networks* (FFNN), *recurrent neural networks* (RNN), and *long short-term memory* (LSTM) models. Further, this thesis explores the application of combining GARCH-type models and DL architectures in a hybrid approach. Thus, this thesis aims to answer the following research questions:

RQ1: How do deep learning architectures compare with ARIMA in forecasting one-step ahead returns of the S&P 500?

RQ2: How do deep learning architectures compare with GARCH-type models in forecasting the one-step ahead volatility of the S&P 500?

To successfully implement this comparative study and to address these research questions, the remainder of this thesis is organised as follows: chapter 2 provides a theoretical background on financial time series data, econometric time series models and DL architectures, and the intricacies of model performance evaluation, before concluding on the theoretical basis by deriving the seven underlying hypotheses of this thesis. Building upon their theoretical introduction, chapter 3 outlines the technical aspects of the econometric models (ARIMA, ARCH, and GARCH) and the DL architectures (FFNN, RNN, and LSTM) employed in this comparative study. Chapter 4 focuses on the methodology, describing the data and how its collection and preprocessing was conducted to fit the necessary requirements outlined in the previous two chapters. Moreover, this chapter outlines the validation approach, the models' computational implementation using R, and how the model performances are evaluated and compared, following the guidelines highlighted in chapter 2. Chapter 5 splits the model evaluation into two parts: (1) one-step ahead daily return forecasts using ARIMA, FFNN, RNN, and

LSTM and (2) volatility forecasts using ARCH/GARCH-type models, FFNN, RNN, and LSTM. Finally, chapter 6 discusses the reasons behind the model performances, linking them back to the theoretical underpinning of chapter 2 and 3. Moreover, this chapter revisits the hypotheses, answers the research questions, and outlines limitations and future research directions. Chapter 7 concludes on this thesis.

2 Literature Review and Theoretical Background

This chapter produces the theoretical groundwork for this comparative study and narrows down its scope by (1) describing the key characteristics of its context: financial time series data, (2) providing information on the two objects under comparison in this context: econometric time series models and DL architectures, and (3) discussing how different models' performances are evaluated and compared. Finally, based on this theoretical foundation, this chapter concludes by stating seven falsifiable hypotheses, which are later inspected to answer the proposed research questions (see chapter 6).

2.1 Financial Time Series Data

The complexity of modelling financial time series problems mainly stems from the existence of statistical regularities (often referred to as *stylised facts*), which, in financial time series data, are not only common across a wide range of instruments, markets, and time periods but also very hard to reproduce artificially with statistical models (Cont, 2001; Francq & Zakoian, 2010). As this comparative study focuses on forecasting returns and volatility of financial time series, this section covers not only its stylised facts but provides definitions for both metrics.

Returns are defined as follows: Given the daily closing price $Close_t$, holding an asset for one period from $t - 1$ to t results in a simple return

$$R_t = \frac{Close_t}{Close_{t-1}} - 1 = \frac{Close_t - Close_{t-1}}{Close_{t-1}}. \quad (2.1)$$

However, using continuously compounded returns (or *log returns*) instead of simple returns is advantageous because one can calculate multiperiod log returns by simply summing the corresponding one-period log returns, a property that does not hold for simple returns (Tsay, 2010), and they are easier to compare between assets as they do not depend on monetary units (Francq & Zakoian, 2010). The log return r_t is simply defined as the natural logarithm of the simple return R_t , thus

$$r_t = \ln(1 + R_t) = \ln \frac{Close_t}{Close_{t-1}} = p_t - p_{t-1}, \quad (2.2)$$

where $p_t = \ln(Close_t)$ (Tsay, 2010). Onwards from here, this thesis simply uses the term *returns* when referring to log returns for simplicity and conciseness (unless stated otherwise).

In financial time series analysis, *stationarity* constitutes its foundation (Tsay, 2010) and has important implications for the modelling of financial time series (Wu et al., 1996), as identifying statistical properties over time would be pointless if they do not remain stable over time (Cont, 2001). In general, stationarity means that a time series is not depended on the time (Hyndman & Athanasopoulos, 2014) and can be subdivided into *strict* and *weak* stationarity. A time series r_t is labelled strictly stationary if the joint distribution of r_t is invariant under time shift. On the other hand, it is labelled weakly stationary if both the mean of r_t and the covariance between r_t and $r_{t-\ell}$ (where ℓ is an arbitrary integer), are time invariant (Tsay, 2010).

However, one of financial time series' stylised facts is that, unlike many time series in natural science, price series often display *non-stationarity* (Francq & Zakoian, 2010, Wu et al., 1996). Crucially, the corresponding return series tend to fulfil the weak stationarity requirement and are thus compatible for financial time series analysis (Francq & Zakoian, 2010). In practice, weak stationarity allows practitioners to make assumptions concerning future observations, for example, forecasting future data points (Tsay, 2010). To showcase this, Figure 1 highlights both the non-stationarity of prices series and the weak stationarity of return series by presenting the daily closing prices (left) and the daily returns (right) of Apple from January 2021 to March 2025.

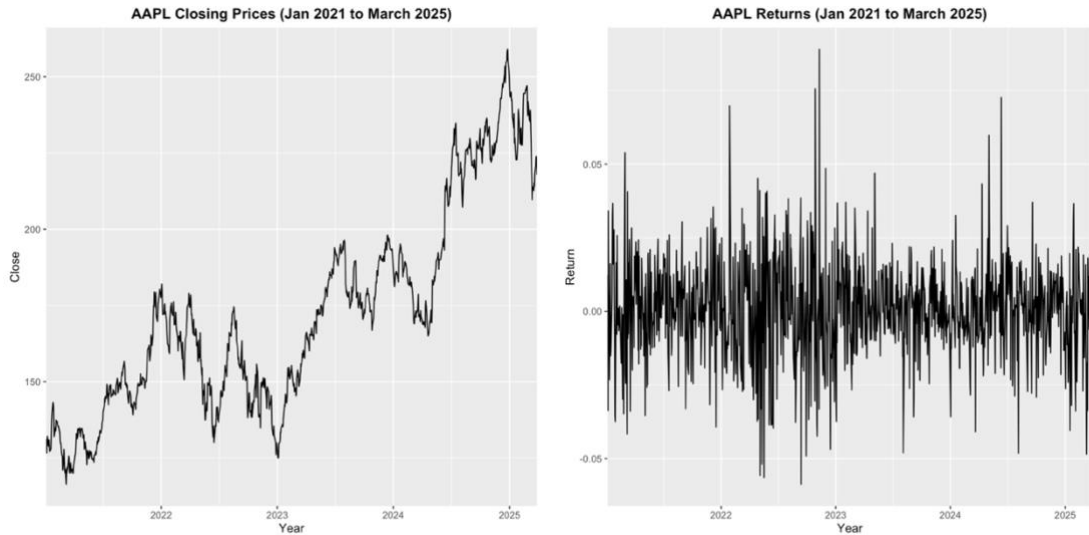


Figure 1: Daily Closing Prices (L) and Returns (R) of Apple from Jan 2021 to Mar 2025.

In financial time series analysis, *autocorrelation* generalises the concept of correlation and provides information about the linear dependence between, for example, r_t and past values $r_{t-\ell}$ by using the autocorrelation function (ACF). Here, the correlation between

r_t and $r_{t-\ell}$ is commonly called the lag- ℓ autocorrelation of r_t (Tsay, 2010) and denoted as

$$\rho_\ell = \frac{\text{Cov}(r_t, r_{t-\ell})}{\sqrt{\text{Var}(r_t)\text{Var}(r_{t-\ell})}} = \frac{\text{Cov}(r_t, r_{t-\ell})}{\text{Var}(r_t)} = \frac{\gamma_\ell}{\gamma_0}. \quad (2.3)$$

Return series typically display only small autocorrelations, making them close to white noise (Francq & Zakoian, 2010) and often insignificant, except for *microstructure effects* when using very small intraday time scales (Cont, 2001). The squared returns (or absolute returns) of a price series, on the other hand, are typically strongly autocorrelated (Francq & Zakoian, 2010) and the ACF of a squared return series decays slowly as a function of the time lag ℓ , which is sometimes interpreted as a sign of long-range dependence (Cont, 2001). To showcase this, Figure 2 displays the ACFs of the returns (left) and the squared returns (right) of Apple's closing price, highlighting their small and strong autocorrelations, respectively.

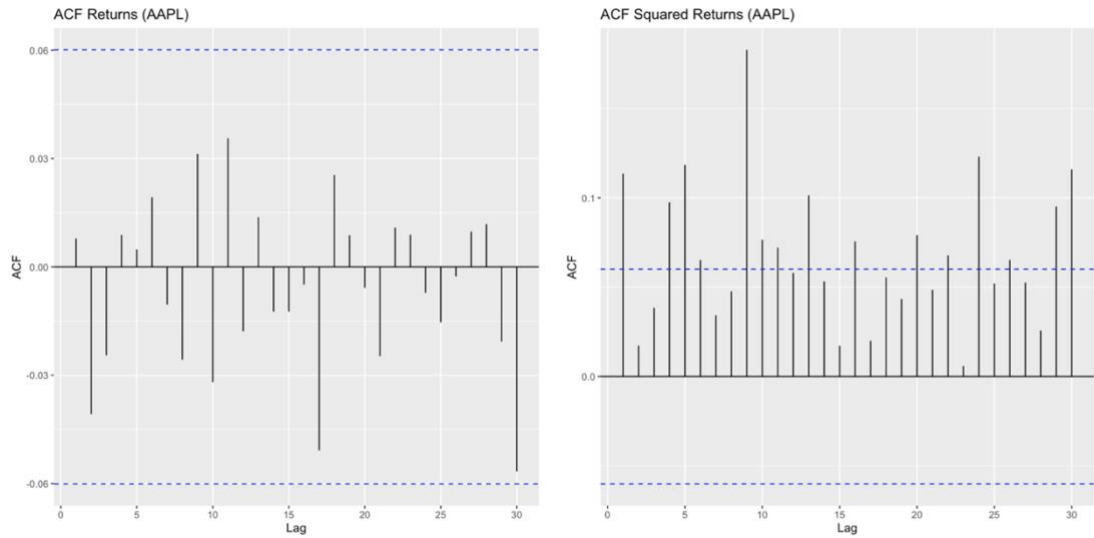


Figure 2: Correlogram of Returns (L) and Squared Returns (R) of Apple in Q1 2021.

When looking at daily returns, their empirical distribution does not follow a Gaussian distribution, as the density sharply peaks at zero and decreases more slowly towards zero, displaying heavy tails (or *leptokurticity*) (Cont, 2001; Francq & Zakoian, 2010). In practice, a leptokurtic distribution means that a random sample of such a time series has a higher likelihood to contain extreme values (Tsay, 2010). The shape of the distribution is not the same at different time intervals, as increasing the time intervals tends to decrease the leptokurticity and the empirical distributions starts to increasingly resemble a Gaussian distribution (*aggregational Gaussianity*) (Cont, 2001; Francq & Zakoian, 2010). Moreover, daily return series display a high degree of variability and the presence

of irregular bursts of volatility (*intermittency*) as well as an asymmetry of gains and losses, i.e., one can observe large decreases but not equally large increases (Cont, 2001). Similarly, positive and negative returns tend to affect volatility differently, given that negative returns tend to increase volatility more than positive ones of the same magnitude, displaying the existence of so-called *leverage effects* (Francq & Zakoian, 2010; Tsay, 2010). Thus, there is a negative correlation between volatility and returns (Cont, 2001).

A key feature distinguishing financial time series analysis from other statistical analyses is that volatility, in contrast to returns, is not directly observable, as there is only one return observations in a trading day (Tsay, 2006; Tsay, 2010). In financial time series, *volatility* constitutes the conditional standard deviation (Tsay, 2010) or instantaneous standard deviation (Carmona, 2014) of an asset and has many applications, for instance, as an important factor in options trading or as a simple approach to calculating value at risk (Tsay, 2010). One of the key motivations behind modelling volatility lies in return series displaying that their variance changes over time; a phenomenon which is called *heteroscedasticity* (Carmona, 2014). Moreover, volatility displays a positive autocorrelation over several days (Cont, 2001), indicating how turbulent periods (high volatility) and quiet periods (low volatility) tend to appear in clusters (*volatility clustering*) (Francq & Zakoian, 2010; Tsay, 2010). While these subperiods are recurrent, they are not periodic (Francq & Zakoian, 2010). Furthermore, calendar effects can be worth monitoring because seasonalities, for example, the day of the week or the proximity of holidays, may significantly affect the returns (*seasonality*). For instance, following a market closure, volatility tends to increase, reflecting the information cumulated during the break (Francq & Zakoian, 2010). Typically, volatility evolves over time in a continuous manner (i.e., volatility jumps are rare) and does not diverge to infinity. Instead, volatility varies in a fixed range and thus is, statistically speaking, often stationary (Tsay, 2010). Figure 3 displays the daily returns of Microsoft from January 1990 to March 2025, highlighting that (1) volatility changes over time (heteroscedasticity), (2) turbulent periods cluster (volatility clustering), and (3) volatility remains in a fixed range (stationarity).

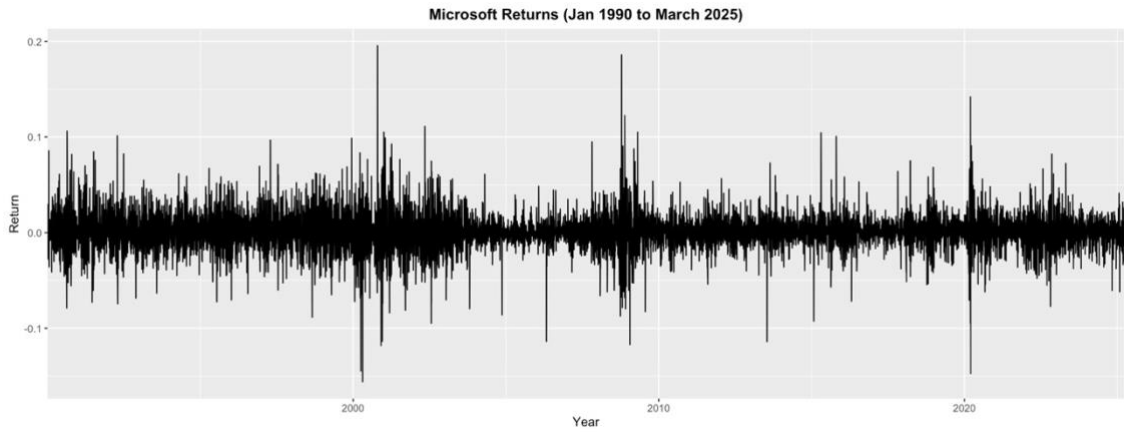


Figure 3: Returns of Microsoft from January 1990 to March 2025.

Some econometric models are specifically designed to capture these characteristics and fix the weaknesses of existing ones which may not capture them accurately, for instance, the exponential GARCH (EGARCH) model to capture the asymmetry in volatility stemming from large positive and negative returns (Tsay, 2010). Nonetheless, volatility's unobservability makes evaluating the forecasting performances of conditional heteroscedastic models difficult. Chapter 2.5 discusses the challenges of evaluating volatility forecasts in greater detail and examines possible proxies.

2.2 Econometric Time Series Models

Traditional econometric time series models aim to tackle these stylised facts. In accordance with the thesis' scope, this section provides a comprehensive overview of two primary families of econometric models: (1) those for the conditional mean (ARIMA), and (2) those for the conditional variance (ARCH/GARCH), outlining their historical developments, core purposes, and limitations. Chapter 3.1 builds upon this subchapter and later provides more technical descriptions of ARIMA, ARCH, and different GARCH-type models. Onwards from here, this thesis simply uses the acronyms ARIMA, ARCH, and GARCH without appending the word "model" for simplicity and conciseness (unless stated otherwise or when needed for clarity).

To forecast non-stationary time series, which are common in economics and finance, the ARIMA was introduced, whose systematic approach revolves around combining autoregressive (AR) and moving average (MA) terms (Box et al., 1994). In the notation $ARIMA(p, d, q)$, p and q refer to the respective AR and MA terms, while d refers to the differencing, i.e., how often a non-stationary time series had to be differenced to be transformed into a stationary one (Tsay, 2010). Further, when building an ARIMA, it is

emphasised to create the simplest possible model that adequately describes the underlying data, a concept known as parsimony. Due to their subsequent simplicity and both terms having clear statistical interpretations, ARIMA are typically easily interpretable (Box et al., 1994). However, a critical limitation of ARIMA lies in their underlying assumption of constant one-period forecast variance, or *homoscedasticity*. Engle (1982) demonstrated that this assumption is systematically violated in financial time series, where, as previously outlined, variance changes over time (heteroscedasticity).

To address this, Engle's (1982) ARCH formalises volatility's dependence on the past, highlighting its predictability. Moreover, the ARCH has the capabilities to provide forecasts which resemble volatility clustering (Tsay, 2010). Accurately modelling and forecasting volatility is crucial, since it constitutes a key variable in many areas of finance, for example, in asset pricing models and dynamic hedging strategies (Bollerslev et al., 1992). However, ARCH tend to need many parameters to capture the persistent nature of volatility. In response, Bollerslev (1986) introduced the GARCH as a more parsimonious and effective extension to the ARCH.

Despite its improvements, the standard GARCH still encounters notable weaknesses. For example, it fails to account for leverage effects, as the model responds equally to positive and negative innovations. Further, its ability to capture extreme market movements is often insufficient, since its tail behaviour tends to remain too short even with standardised Student-t innovations (Tsay, 2010)

The inability of the standard GARCH to capture the asymmetry of volatility led to the development of numerous extensions (Francq & Zakoian, 2010). To specifically address the leverage effect commonly seen in economics and finance, prominent models were introduced such as the EGARCH (Nelson, 1991) and the threshold GARCH (TGARCH) (Zakoian, 1994). In their empirical review, Poon & Granger (2003) confirm that volatility in financial markets is indeed forecastable and further state econometric models' strong performances. While GARCH outperforms ARCH, both typically tend to be outperformed by models incorporating the asymmetries of volatility (Poon & Granger, 2003). Overall, econometric models are grounded in strong statistical theory and successfully capture several key stylised facts while maintaining a high level of interpretability.

Nonetheless, these models may be overly simplistic and can thus fail to capture real-world complexities that financial data may contain, for example, due to relying on specific assumptions about the distribution of the innovations. These challenges, particularly the potential inability to capture real-world complexities, motivate the exploration of more flexible approaches. Thus, the following subchapter turns to DL approaches to model complex patterns without pre-specifying their functional form.

2.3 Deep Learning Approaches

Following on from econometric models' limitations, this thesis now shifts its focus to DL approaches designed to capture the previously mentioned real-world complexities. In accordance with this comparative study's scope, this section introduces the three underlying DL architectures: the FFNN, RNN, and LSTM model. Onwards from here, this thesis simply uses the acronym LSTM without appending "model" or "network" for simplicity and conciseness (unless stated otherwise or when needed for clarity). The focus lies on their core ideas, strengths, and weaknesses in the context of financial time series analysis. Building upon this, chapter 3.2 later provides more technical descriptions of the models.

First, DL constitutes a class of adaptable models designed to learn complex, non-linear patterns directly from the data, with their unique characteristics making them well-suited for forecasting tasks (Zhang et al., 1998). In finance, DL methods can outperform standard methods in forecasting problems involving large data sets, for example, designing and pricing securities, constructing portfolios, and risk management, by detecting and exploiting complex interactions in the data (Heaton et al., 2017).

A FFNN, or *multilayer perceptron* (MLP), can be understood as a non-linear extension of traditional statistical models like linear regression, where layers of neurons with activation functions automatically learn complex relationships in the data (James et al., 2021). Generally, it constitutes a foundational DL model that approximates a function f by mapping an input x to an output y through a series of composed functions (Goodfellow et al., 2016). The primary strength of FFNNs lies in their ability to model complex, non-linear relationships without prior assumptions about the data, as a simple FFNN with one hidden layer can approximate any continuous function to an arbitrary degree of accuracy (Cybenko, 1989; Hornik et al., 1989). While this makes them a powerful tool in forecasting, they require more data and computational power, are prone

to overfitting due to their typically large parameter set, lack interpretability due to their black-box nature, and may not outperform linear statistical models when the underlying process is linear with little disturbance (Zhang et al., 1998). Moreover, the key limitation of the FFNN architecture for time series analysis is that it is memoryless, as each input is processed independently, meaning the model cannot capture temporal dependencies or retain information from past observations in a sequence (Goodfellow et al., 2016). This architectural shortcoming was the primary motivation for the development of RNNs (Elman, 1990).

To address this, the RNN's architecture incorporates a recurrent loop as a hidden state where the network's output from the previous time step is fed as an input to the current time step, effectively allowing the network to retain information (Elman, 1990). This allows RNNs to process sequential data and capture temporal dependencies (Brownlee, 2018; Goodfellow et al., 2016; James et al., 2021) and vastly improves their learning capabilities and performance (Haykin, 2009). Moreover, RNNs do not learn a new set of parameters at each time step, as they employ parameter sharing, which increases their efficiency and ability to generalise temporal patterns regardless of when they occur in the data (Goodfellow et al., 2016; James et al., 2021). However, as sequences get longer, RNNs suffer from an exploding or vanishing gradient, i.e., the network struggles to propagate error signals back through time, making it very difficult to learn long-range dependencies (Goodfellow et al., 2016; Hochreiter & Schmidhuber, 1997).

In response to the exploding/vanishing gradient problem, the LSTM network was introduced as a specialised type of RNN, which is designed to overcome the long-term dependency problem by replacing the simple recurrent neuron with a more complex memory cell (Hochreiter & Schmidhuber, 1997). This cell has a unique architecture containing a cell state, which acts as a long-term memory, and three distinct gating mechanisms (forget, input, and output gate) to regulate its information flow. Together, these characteristics allow the LSTM to selectively add, remove, and output information, allowing it to learn and retain context over long sequences thus mitigating the exploding/vanishing gradient problem (Goodfellow et al., 2016). Its ability to capture long-range dependencies constitutes the LSTM's primary strength. This has established LSTMs as a benchmark and researchers' preferred choice for financial time series forecasting (Sezer et al., 2020), highlighting their suitability for the domain (Fischer & Krauss, 2018). However, this intricate architecture also introduces weaknesses, as, for

example, the increased computational complexity and the larger number of parameters make LSTMs tedious to train and optimise (James et al., 2021).

Overall, DL architectures can add value to financial time series analysis in different ways. While FFNNs and RNNs introduce non-linearity and add memory, respectively, LSTMs additionally provide reliable long-term memory. Nonetheless their high complexity is accompanied but low interpretability, highlighting the trade-off between model interpretability and prediction accuracy (James et al., 2021).

2.4 Hybrid Approaches

While both econometric and DL models have their strengths and weaknesses, neither constitute universal models which are suitable for all circumstances. For instance, while ARIMAs struggle with complex non-linear problems, applying FFNNs to model linear problems often yields mixed results (Zhang, 2003). Since, both model classes have different capabilities to capture linear and non-linear patterns, combining both can allow to model both linear and non-linear patterns effectively, improving forecasting accuracy (Pai & Lin, 2005). Studies on hybrid approaches combining both model classes show promising results (e.g., Kakade et al., 2022; Kim & Won, 2018; Ramos-Pérez et al., 2019). Typically, time series are composed of a linear and a non-linear component. Thus, after modelling their linear component with an econometric model, the model's residuals contain only the non-linear component, which the model could not capture (Zhang, 2003). If there are uncaptured autocorrelations in the econometric model's residuals, feeding them as input into a DL model, can allow for capturing the non-linear component of a time series (Pai & Lin, 2005). The simplified process of combining both model classes tends to be organised as follows: A time series y_t can be represented by combining its linear component L_t and its non-linear component N_t , so that

$$y_t = L_t + N_t. \quad (2.4)$$

When modelling L_t with an econometric model, its residuals e_t are presented by the difference between the estimated linear component $\hat{L}_t$ and the true value y_t , so that

$$e_t = y_t - \hat{L}_t \quad (2.5)$$

Next, using a DL model with n input nodes, the nonlinear relationships can be discovered, as it models the residuals e_t as function of its n past values

$$e_t = f(e_{t-1}, e_{t-2}, \dots, e_{t-n}) + \varepsilon_t, \quad (2.6)$$

where ε_t represents a random error. Denoting (2.5) as the estimated non-linear component $\hat{N}_t$, both estimated components can be combined as the hybrid model's forecast

$$\hat{y}_t = \hat{L}_t + \hat{N}_t. \quad (2.7)$$

Essentially, these two steps combine the strengths of econometric and DL models, allowing to capture both the linear and the nonlinear component of time series (Zhang, 2003).

2.5 Model Performance Evaluation

Evaluating model performance is a critical component of any comparative study and forecasting task. This subchapter outlines the key steps of this process, introduces different evaluation metrics, and presents solutions for the previously outlined challenges regarding the unobservability of volatility in financial time series analysis.

Despite model evaluation typically being performed once the corresponding models have been implemented, one of its most critical parts must be completed beforehand. To accurately assess models' performance, they should not be evaluated on the same data they have been trained on, as they must be able to withstand the robustness of being tested on unseen data, a test design much closer to reality (Poon & Granger, 2003). Hence, when assessing models' performances, the available data is commonly separated into two portions, *training* and *test* data, where the former is used to estimate the model and the latter is used to evaluate its accuracy (Hyndman & Athanasopoulos, 2021). Occasionally, the literature refers to this as in-sample and out-of-sample data. However, this thesis uses the terms training and test data for simplicity and conciseness (unless stated otherwise).

Models that perform well on the training data do not necessarily perform well on the test data due to a phenomenon known as *overfitting*, i.e., following the noise too closely (James et al., 2021; Hyndman & Athanasopoulos, 2021). Hyndman & Athanasopoulos (2021) further emphasise that, while a perfect fit can always be obtained by using enough parameters, overfitting is just as bad as failing to identify a systematic pattern (*underfitting*). While more flexible models (models with more parameters) typically perform better on the training data, less flexible ones may still perform better on the test data. Hence, there is a trade-off between having more complex models, whose estimates

tend to vary more when fed different data (i.e., having higher *variance*), and having simpler models who may fail to sufficiently capture real-world complexities (i.e., having higher *bias*). This phenomenon is commonly known as the *bias-variance trade-off* (James et al., 2021).

When assessing models' performances, popular evaluation metrics include, for example, *mean squared error* (MSE), *mean absolute error* (MAE), and *mean absolute percentage error* (MAPE) (Poon & Granger, 2003). Denoting the forecast error simply as $e_i = y_i - \hat{y}_i$, where y_i and $\hat{y}_i$ constitute the true value and the model's prediction, respectively, the MSE and MAE can be denoted as

$$MSE = \overline{mean}(e_i^2) \text{ and} \quad (2.8)$$

$$MAE = \overline{mean}(|e_i|). \quad (2.9)$$

Further, letting the percentage error be $p_i = 100e_i/\hat{y}_i$, the MAPE is denoted as

$$MAPE = \overline{mean}(|p_i|). \quad (2.10)$$

In comparison to scale-dependent metrics like MSE and MAE, percentage metrics like MAPE have the advantage of being scale-independent and are therefore often used to compare forecast performance between different datasets. However, they have the disadvantage of being infinite, undefined or having extreme values, when y_i is (close to) zero (Hyndman & Athanasopoulos, 2014).

In volatility forecasting, MSE constitutes the standard measure to assess the training accuracy, while *Quasi-Likelihood* (QLIKE) is commonly used in addition to MSE to assess the test accuracy (Bollerslev et al., 2016). It can be denoted as

$$QLIKE = \frac{\sigma_t^2}{\hat{\sigma}_t^2} - \log\left(\frac{\sigma_t^2}{\hat{\sigma}_t^2}\right) - 1, \quad (2.11)$$

where $\hat{\sigma}_t^2$ is the forecasted volatility and σ_t^2 is the actual volatility at time t .

However, as mentioned in chapter 2.1, volatility cannot be observed in financial time series data. Before high frequency data became available, it was common practice to circumvent this issue by using the squared returns as proxy for the daily volatility (Poon & Granger, 2003). But since daily squared returns constitute a very noisy volatility estimator (Patton, 2011), the empirical findings of studies using this approach should be interpreted with extra caution (Poon & Granger, 2003). Thus, evaluating and comparing

volatility forecasts relies on methods such as using a volatility proxy (Patton & Sheppard, 2009). For instance, in options markets, the Black-Scholes formula can be used to obtain the implied volatility. However, this approach might yield results that differ from the actual volatility and tend to be larger than the estimates of GARCH-type models (Tsay, 2010). Moreover, range-based approaches are commonly used for volatility estimation (Alizadeh et al., 2002). As such, Parkinson's (1980) extreme value method uses the highest and lowest prices of a time interval to estimate its volatility. Compared to squared returns, their volatility estimator provides two-and-a-half to five times better results and can be denoted as

$$\widehat{\sigma}_P^2 = \frac{1}{n} \sum_{i=1}^n \frac{1}{4 \ln 2} \ln \left(\frac{High_i}{Low_i} \right)^2. \quad (2.12)$$

After its introduction, this estimator has been extended and improved by combining the high-low range with the opening and closing prices (Alizadeh et al., 2002). For instance, in contrast to Parkinson (1980), Rogers & Satchell (1991) provide a volatility estimator that is unbiased regardless of the drift and can be denoted as

$$\widehat{\sigma}_{RS}^2 = \frac{1}{n} \sum_{i=1}^n \ln \left(\frac{High_i}{Close_i} \right) \ln \left(\frac{High_i}{Open_i} \right) + \ln \left(\frac{Low_i}{Close_i} \right) \ln \left(\frac{Low_i}{Open_i} \right). \quad (2.13)$$

However, Yang & Zhang (2000) state that it is impossible to have single-period volatility estimators which are independent of the drift and the opening jump. Hence, they introduce an extension to Rogers & Satchell (1991) based on data of multiple periods. Their volatility estimator can be denoted as

$$\widehat{\sigma}_{YZ}^2 = \widehat{\sigma}_O^2 + k \widehat{\sigma}_C^2 + (1 - k) \widehat{\sigma}_{RS}^2, \quad (2.14)$$

where $k_0 = \frac{0.34}{1.34 + \frac{n+1}{n-1}}$, while $\widehat{\sigma}_O^2$ and $\widehat{\sigma}_C^2$ are the estimated volatilities of the opening and closing prices denoted as

$$\widehat{\sigma}_O^2 = \frac{1}{n-1} \sum_{i=1}^n (o_i - \bar{o})^2 \text{ and} \quad (2.15)$$

$$\widehat{\sigma}_C^2 = \frac{1}{n-1} \sum_{i=1}^n (c_i - \bar{c})^2, \quad (2.16)$$

respectively (Yang & Zhang, 2000). Table 1 provides an overview of the different estimators.

Estimator	Uses	Notes	Citation
Squared Returns	Close	Very noisy; common before high-frequency data became available	Poon & Granger (2003), Patton (2011)
Parkinson	High, Low	No drift; no opening jumps	Parkinson (1980)
Rogers-Satchell	Open, High, Low, Close	Unbiased under drift; no opening jumps	Rogers & Satchell (1991)
Yang-Zhang	Open, High, Low, Close	Robust to drift and opening jump; requires multiple time periods	Yang & Zhang (2000)

Table 1: Common Volatility Estimators.

2.6 Hypotheses

Based on the literature and this thesis' scope, seven hypotheses are derived.

First, more complex models like FFNN do not necessarily outperform simpler models when the underlying process is linear and with little disturbance (Zhang et al., 1998). Now, financial return series typically display only small autocorrelations and tend to resemble white noise (Francq & Zakoian, 2010), except for microstructure effects when using very small intraday time scales (Cont, 2001). Hence, when forecasting one-step ahead daily returns for the S&P 500 without using intraday data, this thesis does not expect more complex DL models to outperform econometric time series models:

H1: More complex models (FFNN, RNN, and LSTM) do not outperform econometric time series models (ARIMA) on MSE and MAE when forecasting one-step ahead daily returns for the S&P 500.

Second, due to returns series typically resembling white noise (Francq & Zakoian, 2010), this thesis does expect neither complex DL models nor econometric time series models to significantly outscore the historic mean:

H2: Both econometric time series models (ARIMA) and more complex models (FFNN, RNN, and LSTM) struggle to outperform the historic mean on MSE and MAE when forecasting one-step ahead daily returns for the S&P 500.

Third, since the GARCH was introduced as a more parsimonious and effective extension to the ARCH (Bollerslev, 1986) and since it typically outperforms ARCH (Poon & Granger, 2003), this thesis expects a simple GARCH(1,1) model to outperform even a

more complex ARCH when forecasting one-step ahead daily volatility estimates for the S&P 500:

H3: A standard GARCH(1,1) model outperforms a more complex ARCH model on MSE and QLIKE when forecasting one-step ahead daily volatility estimates for the S&P 500.

Fourth, since GARCH-type models which incorporate the asymmetries of volatility typically outperform the standard GARCH (Poon & Granger, 2003), this thesis expects the EGARCH and TGARCH to outperform a GARCH(1,1) model when forecasting one-step ahead daily volatility estimates for the S&P 500:

H4: GARCH-type models which incorporate the asymmetries of volatility (EGARCH and TGARCH) outperform a standard GARCH(1,1) model on MSE and QLIKE when forecasting one-step ahead daily volatility estimates of the S&P 500.

Fifth, in contrast to return series, their squared returns are typically strongly autocorrelated (Francq & Zakoian, 2010), which can be interpreted as a sign of long-range dependencies (Cont, 2001). Despite squared returns constituting a very noisy volatility estimator (Patton, 2011; Poon & Granger, 2003), this suggests the existence of patterns and dependencies in the data, for instance, volatility clustering (e.g., Francq & Zakoian, 2010; Tsay, 2010). Moreover, volatility is generally forecastable (Engle, 1982; Poon & Granger, 2003). Hence, this thesis expects both econometric time series models and DL models to outscore benchmarks like the historic mean and the naïve method:

H5: Econometric time series models and DL approaches both outscore the historic mean and the naïve method on MSE and QLIKE when forecasting one-step ahead daily volatility estimates for the S&P 500.

Sixth, even though DL models are well-suited for forecasting tasks (Zhang et al., 1998) by detecting and exploiting complex interactions in financial data (Heaton et al., 2017), since FFNNs lack memory (Goodfellow et al., 2016; Elman, 1990) and RNNs struggle to learn long-range dependencies (Goodfellow et al., 2016; Hochreiter & Schmidhuber, 1997), this thesis does not expect a FFNN or RNN to outperform GARCH-type models when forecasting one-step ahead daily volatility estimates for the S&P 500:

H6: Neither FFNN nor RNN outperform econometric time series models (ARCH and GARCH-type models) on MSE and QLIKE when forecasting one-step ahead daily volatility estimates for the S&P 500.

Seventh, since the LSTM was designed to overcome the long-term dependency problem of the RNN (Hochreiter & Schmidhuber, 1997), this thesis expects the LSTM to outperform GARCH-type models when forecasting one-step ahead daily volatility estimates for the S&P 500:

H7: A LSTM outperforms econometric time series models (ARCH and GARCH-type models) on MSE and QLIKE when forecasting one-step ahead daily volatility estimates for the S&P 500.

Chapter 6 revisits these hypotheses to answer the previously proposed research questions.

3 Models

This chapter provides detailed descriptions of the models used in this comparative study and is divided into two subchapters focusing on econometric time series models (3.1) and the DL architectures (0).

3.1 Econometric Time Series Models

In financial times series analysis, econometric time series models constitute key tools for modelling and forecasting financial data such as return series or their volatility. The following subchapter displays the ARIMA (0), ARCH (3.1.2), and GARCH (3.1.3) models, respectively, in greater detail.

3.1.1 Autoregressive Integrated Moving Average (ARIMA)

ARIMA models are widely used for time series forecasting (Hyndman & Athanasopoulos, 2014) and were popularised by Box & Jenkins (1970). They combine autoregressive (AR) and moving average (MA) models with differencing handling time series' non-stationarity (Box et al., 1994).

In finance, the basic idea behind AR models is to predict the return r_t by using a linear combination of its past values (Hyndman & Athanasopoulos, 2014; Tsay, 2010). Thus, an $AR(p)$ assumes that the return r_t can be explained as a function of its p past values (Shumway & Stoffer, 2025) and can thus be denoted as

$$r_t - \mu = \sum_{i=1}^p \phi_i r_{t-i} + e_t, \quad (3.1)$$

Where μ is the mean of $\{r_t\}$, $\{e_t\}$ is a white noise series with mean zero and variance σ_t^2 , and ϕ constitutes the parameters. Defining the backshift operator B as

$$B^i r_t = r_{t-i}, \quad (3.2)$$

where its inverse B^{-1} constitutes the forward-shift operator with

$$B^{-1} r_{t-1} = B^{-1} B r_t = r_t \text{ and} \quad (3.3)$$

$$B^{-1} B = 1, \quad (3.4)$$

allows for rewriting the $AR(p)$ model (3.1) as

$$\phi_p(B)(r_t - \mu) = e_t, \quad (3.5)$$

where

$$\phi_p(B) = 1 - \sum_{i=1}^p \phi_i B^i. \quad (3.6)$$

Instead of using past values of a time series $\{r_t\}$, a MA model predicts the return r_t by using a linear combination of its past errors (Hyndman & Athanasopoulos, 2014). Thus, a MA(q) model can be denoted as

$$r_t = \mu + \sum_{i=1}^q \theta_i e_{t-i} + e_t, \quad (3.7)$$

where θ constitutes the parameters of the MA(q) model, or by using the backshift operator B (3.2), analogous to rewriting AR(p) in (3.5), as

$$r_t = \mu + \theta_q(B)e_t, \quad (3.8)$$

where

$$\theta_q(B) = 1 + \sum_{i=1}^q \theta_i B^i. \quad (3.9)$$

Now, a simple MA(1) process

$$r_t = \mu + \theta_1(B)e_t, \quad (3.10)$$

with $\theta_1 < |1|$ can be written as an AR(∞), thus constituting a non-parsimonious representation of an AR model. Similarly, a stationary AR(1) has a non-parsimonious MA(∞) representation. Thus, in practice, it can be necessary to combine both AR and MA components in an autoregressive moving average (ARMA) model (Box et al., 1994). An ARMA(p, q) can thus be denoted by combining (3.5) and (3.8) as

$$\phi_p(B)(r_t - \theta_0) = \theta_q(B)e_t \text{ or} \quad (3.11)$$

when isolating the target variable r_t as

$$r_t = \theta_0 + \phi_p^{-1}(B)\theta_q(B)e_t, \quad (3.12)$$

where the constant term $\theta_0 = \mu(1 - \sum_{i=1}^p \phi_i)$. Setting $p = 0$ or $q = 0$ reduces the ARMA (3.12) to a MA(q) (3.8) or AR(p) model (3.5), respectively.

Now, the integrated ARMA (ARIMA), includes differencing to handle time series' being non-stationary (Shumway & Stoffer, 2025). Let the difference be defined as $\nabla r_t = r_t - r_{t-1}$. Then, by using the backshift operator B (3.2), it can be written as

$$\nabla^d r_t = (1 - B)^d r_t, \quad (3.13)$$

where d indicates how often the series was differenced. Hence, the ARIMA(p, d, q) model can be denoted as

$$\phi_p(B)(1 - B)^d r_t = \theta_q(B)e_t, \quad (3.14)$$

where the $\theta_0 = 0$. Differencing constitutes the conventional approach for handling non-stationarity. In finance, differencing is used for transforming a non-stationary price series into a stationarity one by calculating the returns (eq. 2.2). (Tsay, 2010).

3.1.2 Autoregressive Conditional Heteroscedasticity (ARCH)

The ARCH was proposed by Engle (1982) and constitutes a key approach for modelling conditional heteroscedasticity (also referred to as the ARCH effect) in economic and financial time series. It challenges previous econometric models' implausible assumption of a constant one-period forecast variance (Engle, 1982).

The basic idea of the ARCH is that the shock a_t of a return is serially uncorrelated but dependent, and that this dependence can be described by a quadratic function of the lagged values of a_t . The ARCH(p) model assumes that

$$a_t = \sigma_t \epsilon_t \text{ and } a_t | \mathcal{F}_{t-1} \sim N(0, \sigma_t^2), \quad (3.15)$$

where $\{\epsilon_t\}$ is a sequence of independent and identically distributed (iid) random variables with mean $E(\epsilon) = 0$, $var(\epsilon_t) = 1$. Subsequently, the ARCH (p) model is defined as

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^p \alpha_i a_{t-i}^2, \quad (3.16)$$

where $\alpha_0 > 0$, and $\alpha_i \geq 0$ for $i > 0$ (Carmona, 2014; Tsay, 2010). The model's structure shows that large shocks a_t imply a high volatility σ_t^2 and thus tend to be followed by another large shock. This resembles the volatility clustering typically observed in financial return series (Tsay, 2010).

Considering the unconditional mean and variance of a_t , the former remains zero (3.17) while the latter (3.18) can be obtained as

$$E(a_t) = E[E(a_t | \mathcal{F}_{t-1})] = E[\sigma_t E(\epsilon_t)] = 0, \quad (3.17)$$

$$Var(a_t) = E(a_t^2) = \alpha_0 + \alpha_1 E(a_{t-1}^2), \quad (3.18)$$

where $\mathcal{F}_{t-1}$ denotes the information set available at time $t - 1$ and consists of all linear functions of the past returns, and where $0 \leq \alpha_1 < 1$ (Tsay, 2010).

For the variance to be finite, the condition $\alpha_1 < 1$ must be fulfilled, while a finite fourth moment also requires that $3\alpha_1^2 < 1$ (Engle, 1982). If these conditions are met, the fourth moment can be calculated by

$$E(a_t^4) = \frac{3\alpha_0^2(1+\alpha_1)}{(1+\alpha_1)(1-3\alpha_1^2)}. \quad (3.19)$$

With $3\alpha_1^2 < 1$ and the excess kurtosis of a_t being positive, the tail distribution of a_t is heavier than that of a normal distribution, the shock a_t of an ARCH(p) model is more likely to produce outliers than a normally distributed white noise series (Tsay, 2010).

Despite its advantages, the ARCH has some weakness. For instance, assuming positive and negative shocks to have the same effect on volatility, i.e., failing to capture leverage effects, failing to provide any new insights for understanding why a financial time series has variations, and even a higher order ARCH being limited in capturing excess kurtosis (Tsay, 2010).

3.1.3 Generalised Autoregressive Conditional Heteroscedasticity (GARCH)

The GARCH as proposed by Bollerslev (1986) constitutes an extension of Engle's (1982) ARCH and allows for a much more flexible lag structure (Bollerslev, 1986). Similarly to the ARCH, the GARCH differentiates between the unconditional and conditional variance and allows for the latter to change over time (Bollerslev, 1986). The GARCH (p, q) model assumes that

$$a_t = \sigma_t \epsilon_t \quad a_t | \mathcal{F}_{t-1} \sim N(0, \sigma_t^2), \quad (3.20)$$

where $\{\epsilon_t\}$ is a sequence of iid random variables with mean $E(\epsilon) = 0$ and $var(\epsilon_t) = 1$, and where $\mathcal{F}_{t-1}$ denotes the information set available at time $t - 1$ and consists of all linear functions of the past returns, and where $0 \leq \alpha_1 < 1$. Subsequently, the GARCH (p, q) model is then defined as

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^p \alpha_i a_{t-i}^2 + \sum_{j=1}^q \beta_j \sigma_{t-j}^2, \quad (3.21)$$

where $\alpha_0 > 0$, $\alpha_i \geq 0$ for $i > 0$, $\beta_j \geq 0$ for $j > 0$, and $\sum_{i=1}^{\max(p,q)} (\alpha_i + \beta_j) < 1$. The latter constraint implies that the unconditional variance of a_t is finite, while its conditional variance σ_t^2 evolves over time (Tsay, 2010) and suffices for wide-sense stationarity (Bollerslev, 1986). When setting $q = 0$, the GARCH($p, 0$) (3.21) reduces to the ARCH(p) (3.16). Furthermore, when setting $p = 0$ and $q = 0$, GARCH(0,0) (3.21)

reduces to white noise (Bollerslev, 1986; Francq and Zakoian, 2010; Tsay, 2010). In Bollerslev (1986) the p order is associated with the GARCH term, while the q order is associated with the ARCH process. In accordance with common textbooks (e.g., Tsay, 2010) and the lectures, in this thesis, the p order is nonetheless associated with the ARCH process, while the q order is associated with the GARCH term (unless stated otherwise). The parameters α_i and β_j are commonly referred to as the ARCH and GARCH parameters, respectively (Tsay, 2010).

The second (3.22) and fourth moment (3.23) can be calculated, respectively, by

$$E(a_t^2) = \frac{\alpha_0}{(1-\alpha_1-\beta_1)} \text{ and} \quad (3.22)$$

$$E(a_t^4) = \frac{3\alpha_0^2(1+\alpha_1+\beta_1)}{(1-\alpha_1-\beta_1)(1-\beta_1^2-2\alpha_1\beta_1-3\alpha_1^2)}. \quad (3.23)$$

Further, the coefficient of kurtosis can be calculated by

$$\kappa = \frac{6\alpha_1^2}{(1-\beta_1^2-2\alpha_1\beta_1-3\alpha_1^2)}. \quad (3.24)$$

with $\kappa > 0$ by assumption. Hence, the GARCH(1,1) model is, similar as the ARCH(p) model, leptokurtic, i.e., heavily tailed (Bollerslev, 1986).

Focusing on the simplest GARCH(1,1) model

$$\sigma_t^2 = \alpha_0 + \alpha_1 a_{t-1}^2 + \beta_1 \sigma_{t-1}^2, \quad (3.25)$$

highlights how large a_{t-1}^2 or σ_{t-1}^2 tend to result in large σ_t^2 , thus reflecting the well-known behaviour of volatility clustering in financial time series. Moreover, using a GARCH(1,1) model and assuming h to be the forecasting origin, 1-step-ahead (3.26) and ℓ -step-ahead forecasts (3.27) can be calculated, respectively, by

$$\sigma_h^2(1) = \alpha_0 + \alpha_1 a_h^2 + \beta_1 \sigma_h^2 \text{ and} \quad (3.26)$$

$$\sigma_h^2(\ell) = \alpha_0 + (\alpha_1 + \beta_1) \sigma_h^2(\ell - 1), \quad (3.27)$$

as a_h and σ_h^2 are known at the time index h (Tsay, 2010).

3.1.4 GARCH-Extensions

Numerous extensions to the GARCH have been proposed to handle the asymmetries of volatility, which constitutes one of the standard GARCH's key weaknesses. First, the EGARCH was proposed by Nelson (1991). The EGARCH(p, q) is defined as

$$\ln(\sigma_t^2) = \alpha_0 + \sum_{i=1}^q \alpha_i g(\eta_{t-1}) + \sum_{j=1}^p \beta_j \ln(\sigma_{t-j}^2), \quad (3.28)$$

where

$$g(\eta_{t-1}) = \theta \eta_{t-1} + \varsigma(|\eta_{t-1}| - E|\eta_{t-1}|), \quad (3.29)$$

and where α_0 , α_i , β_j , θ , and ς are real numbers (Francq & Zakoian, 2010). In (3.28), $g(\eta_{t-1})$ is responsible for handling the asymmetric effects between positive and negative asset returns differently (Tsay, 2010).

Second, the TGARCH was introduced by Zakoian (1994). The TGARCH(p, q) is defined as

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^q (\alpha_i + \gamma_i N_{t-1}) a_{t-i}^2 + \sum_{j=1}^p \beta_j \sigma_{t-j}^2, \quad (3.30)$$

where

$$N_{t-1} = \begin{cases} 1 & \text{if } a_{t-1} < 0 \\ 0 & \text{if } a_{t-1} \geq 0 \end{cases}, \quad (3.31)$$

indicates whether the previous time steps asset return a_{t-1} was negative (Tsay, 2010).

3.2 Deep Learning Models

Neural networks (NN) not only represent the cornerstone of DL (James et al., 2021) but their introduction in the mid 1980s marked the shift of predictive modelling towards computer science and ML (Efron & Hastie, 2017). Since this thesis is based on different types of NNs, the following subchapters now present FFNNs (3.2.1), RNNs (3.2.2), and LSTMs (0), respectively, in greater detail.

3.2.1 Feed-Forward Neural Networks (FFNN)

The general idea of a FFNN is to predict a response Y by taking an input vector of p variables $X = (X_1, X_2, \dots, X_p)$ and building a nonlinear function $f(X)$. What differentiates a FFNN from other methods, e.g., decision trees, is its structure (see Figure 4), consisting of an input layer (the p variables X), one or multiple hidden layers and an output layer (James et al, 2021).

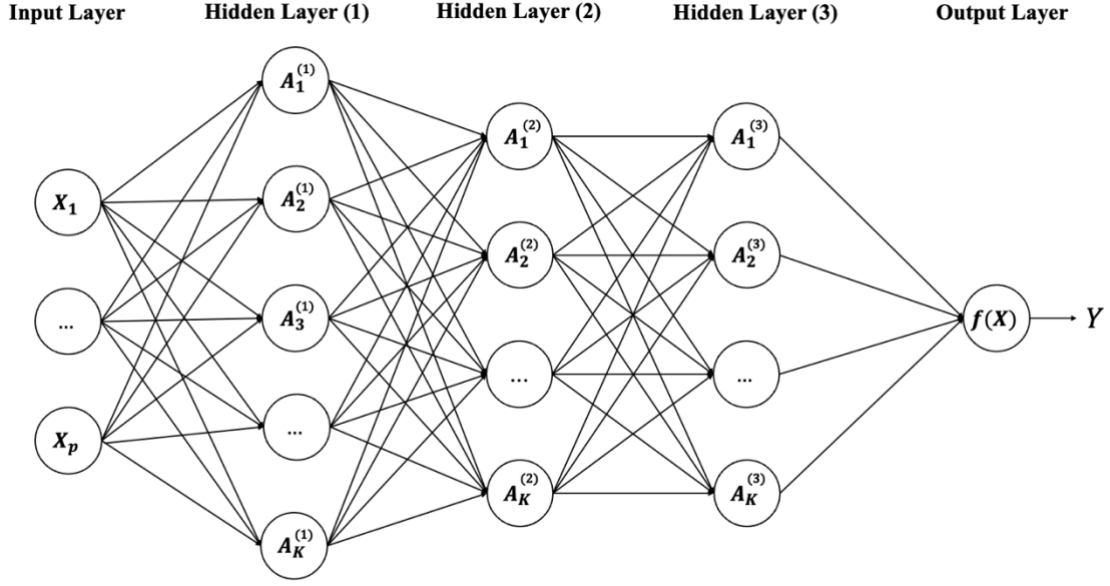


Figure 4: FFNN with Three Hidden Layers following James et al., (2021, p. 409).

The hidden layer's units (also referred to as nodes or neurons) intervene between the external input and the output in some useful manner, as hidden layers allow to extract higher-order statistics from the input (Haykin, 2009). As indicated by the arrows, each input unit feeds forward into each of the K hidden units (James et al., 2021), and as every unit in each layer is connected to every other unit in the adjacent forward layer, the network is said to be fully connected (Haykin, 2009). First, the input units supply respective elements of the input to the hidden units. Then, the hidden units' outputs are used as inputs for to the next layer, and so on for the rest of the network. Typically, each layers' units only have the preceding layer's output. Further, a FFNN with m input units, h_1 hidden units in the first hidden layer, ..., h_i hidden units in the i th hidden layer, and q output units, is referred to as an m - h - q network (Haykin, 2009). The FFNN has the form

$$f(X) = \beta_0 + \sum_{k=1}^K \beta_k g(w_{k0} + \sum_{j=1}^p w_{kj} X_j). \quad (3.32)$$

A FFNN is built in two steps. First, the K hidden units (or activations A_k) are computed as functions of the input units X by

$$A_k = g(w_{k0} + \sum_{j=1}^p w_{kj} X_j), \quad (3.33)$$

with g being the activation function (see below), and w_{k0} and w_{kj} constituting the bias term of the hidden layer and the synaptic weights, respectively. The synaptic weights w_{kj} denote the strength of the connection between an input unit X and a hidden unit A_k (James et al., 2021). Here, summing the bias term and the input signals (multiplied by the

respective weights) is called linear combination (Haykin, 2009). Second, these activations feed into the output layer, resulting in a linear regression model

$$f(X) = \beta_0 + \sum_{k=1}^K \beta_k A_k, \quad (3.34)$$

where all parameters $\beta_0, \dots, \beta_K$ and $w_{10}, \dots, w_{Kp}$ must be estimated from the data. For DL models with more than one hidden layer, the second hidden layer treats the previous hidden layer's activations $A_k^{(1)}$ (3.33) as input, computing the second set of activations

$$A_k^{(2)} = g(w_{k0}^{(2)} + \sum_{j=1}^{K_1} w_{kj}^{(2)} A_j^{(1)}), \quad (3.35)$$

where K_1 denotes the number of nodes in the first hidden layer. The activation function g is specified in advance (James et al., 2021) and limits the amplitude of the units' outputs (Haykin, 2009). The sigmoid activation function is the well-known activation function and constitutes a standard approach (Schmidt-Hieber, 2020) due to its nonlinearity, computational simplicity, sufficient degree of smoothness (Han & Moraga, 1995). Further, the nonlinearity of the activation function is essential for (1) avoiding the model $f(X)$ to collapse into a simple linear model and (2) allowing the model to capture complex nonlinearities and interaction effects (James et al., 2021). The sigmoid activation function is also used in logistic regression to convert linear functions into probabilities and is denoted as

$$g(z) = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}. \quad (3.36)$$

However, modern FFNNs favour the ReLU (rectified linear unit) activation function

$$g(z) = (z)_+ = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{otherwise} \end{cases} \quad (3.37)$$

because of its efficiency and computability (James et al., 2021; Schmidt-Hieber, 2020). ReLU functions are piecewise linear approximations. Typically, nonparametric methods based on piecewise linear approximations cannot capture higher-order smoothness past smoothness index two. However, combining ReLU activation with deep network architectures achieves near minimax rates for arbitrary smoothness of the regression function (Schmidt-Hieber, 2020).

Fitting a FFNN, i.e., estimating its parameters β and ω_k , can be done by solving a nonlinear least squares problem

$$\underset{\{\omega_k\}_1^K, \beta}{\text{minimise}} \frac{1}{2} \sum_{i=1}^n (y_i - f(x_i))^2, \quad (3.38)$$

where $f(x_i)$ is (eq. 3.32). However, minimising this formula is not straightforward, as the problem is nonconvex in its parameters and thus there is a distinct possibility of obtaining a solution which lacks uniqueness and stability (Haykin, 2009; James et al., 2021). Moreover, considering the simplicity of the FFNN displayed in Figure 4, the problems are compounded when fitting more complex models (James et al., 2021). One way to overcome this problem is to expand (3.38) by adding a new term

$$\underset{\{\omega_k\}_1^K, \beta}{\text{minimise}} \frac{1}{2} \sum_{i=1}^n (y_i - f(x_i))^2 + \frac{\lambda}{2} \|\omega\|^2, \quad (3.39)$$

where the squared Euclidean norm $\|\omega\|^2$ is referred to as structural regularization with the scalar λ being called regularisation parameter. Here, λ lies between 0 and ∞ , with 0 and ∞ implying total or no confidence in the trained model, respectively (Haykin, 2009). Aside from imposing penalties on the parameters, the problem can also be overcome through slow learning, i.e., fitting the model in a somewhat slow iterative fashion by using gradient descent and stopping the fitting process when overfitting is detected. For this, it can be considered to rewrite (3.38) to

$$R(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_\theta(x_i))^2, \quad (3.40)$$

where θ represents all parameters in one long vector and f is explicitly depended on them. Next, after guessing all parameters θ^0 and setting $t = 0$, the objective is to find a vector δ which represents a small change in θ , such that $R(\theta^{t+1}) < R(\theta^t)$. After setting $t \leftarrow t + 1$, this process is repeated until equation (3.40) fails to decrease (James et al., 2021). In this training process of deep models, the backpropagation algorithm constitutes the dominant approach to efficiently compute these gradients (Goodfellow et al., 2016).

3.2.2 Recurrent Neural Networks (RNN)

RNNs constitute a family of neural networks for processing sequential data (Goodfellow et al., 2016). What distinguishes the RNN's architecture from the FFNN's architecture is that it contains at least one feedback loop (Haykin, 2009) that allows to use an output of the network from a prior step as an input to automatically learn across sequence data (Brownlee, 2018). This recurrent unit helps RNNs to handle sequential patterns found in data (Vishwas & Patel, 2020) and has a profound impact on their learning capabilities

and performance (Haykin, 2009), as they can scale to much longer sequences than it would be practical for networks without sequence-based specialisation (Goodfellow et al., 2016).

In an RNN, the input object X is a sequence $\{X_1, X_2, \dots, X_L\}$, where L denotes its length. Figure 5 illustrates the structure of a simple RNN with an input-layer sequence X_ℓ , an output-layer sequence O_ℓ providing an output Y , and a hidden-layer sequence $\{A_\ell\}_1^L = \{A_1, A_2, \dots, A_L\}$ (James et al., 2021).

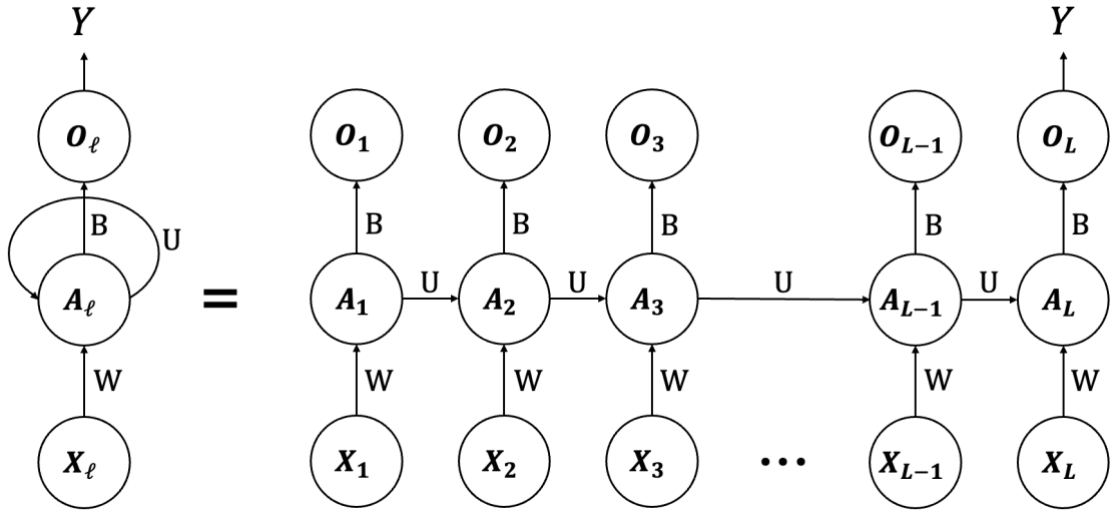


Figure 5: Schematic of a Simple RNN following James et al. (2021, p. 422).

Typically, the sequence is processed one vector X_ℓ at a time. Then, the network updates the activations A_ℓ in the hidden layer by taking as input the vector X_ℓ and the activation vector $A_{\ell-1}$ from the previous step in the sequence. Each A_ℓ feeds into the output layer and produces a prediction Y . The last prediction O_L is the most relevant (James et al., 2021). An important aspect of this process is parameter sharing, i.e., an RNN shares the same weights across several time steps, as it allows to extend and apply the model to examples of different forms (Goodfellow et al., 2016). In Figure 5, parameter sharing is illustrated by W , U , and B representing the shared weights w_{kj} , u_{ks} , and β_k , respectively.

The activation layer

$$A_{\ell k} = g(w_{k0} + \sum_{j=1}^p w_{kj} X_{\ell j} + \sum_{s=1}^K u_{ks} A_{\ell-1, s}), \quad (3.41)$$

and the output O_ℓ are computed as

$$O_\ell = \beta_0 + \sum_{k=1}^K \beta_k A_{\ell k}, \quad (3.42)$$

with g representing the activation function, e.g., sigmoid or ReLU. Further, the loss function

$$(Y - O_L)^2 \quad (3.43)$$

only references the final output O_L , however each element of the input sequence X_ℓ contributes to the final output through the accumulated history of the activations A_ℓ (James et al., 2021). Like fitting a FFNN, an RNN is fitted by minimising

$$\sum_{i=1}^n (y_i - O_{iL})^2, \quad (3.44)$$

where

$$O_{iL} = \beta_0 + \sum_{k=1}^K \beta_k g(w_{k0} + \sum_{j=1}^p w_{kj} X_{iLj} + \sum_{s=1}^K u_{ks} A_{i,L-1,s}). \quad (3.45)$$

Typically, the parameters are estimated by using the backpropagation through time approach (BPTT), which is a gradient descent algorithm that uses a derived chain rule (Vishwas & Patel, 2020).

As the weights are continuously updated, the gradient becomes either too weak or too strong, causing the gradient vanishing/exploding problem (Vishwas & Patel, 2020). This constitutes one of the biggest issues with BPTT and RNNs. An exploding gradient may lead to oscillating weights, while a vanishing gradient makes it difficult to bridge long time lags (Hochreiter & Schmidhuber, 1997).

3.2.3 Long Short-Term Memory (LSTM)

Subsequently, Hochreiter & Schmidhuber (1997) first introduced the LSTM as a faster and more efficient method to address previous RNN architectures' problem of storing information over extended time intervals (as seen above). As a special type of RNN, LSTMs read each time step of an input sequence one at a time, offering native support for sequences (Brownlee, 2018). The general idea of LSTM is to combine the recurrent network architecture with an appropriate gradient-based learning algorithm to overcome the error back-flow problems and learn to bridge time intervals more than 1,000 steps. To achieve this, the gradient-based algorithm enforces constant error flow, hence no vanishing or exploding, through internal states of special units without affecting long-term error flow (Hochreiter & Schmidhuber, 1997).

Structurally, LSTMs are like standard RNNs, as they consist of a chain of repeating modules. However, in LSTMs, the repeating modules are different and interact in a very special way (Olah, 2015) (see Figure 6).

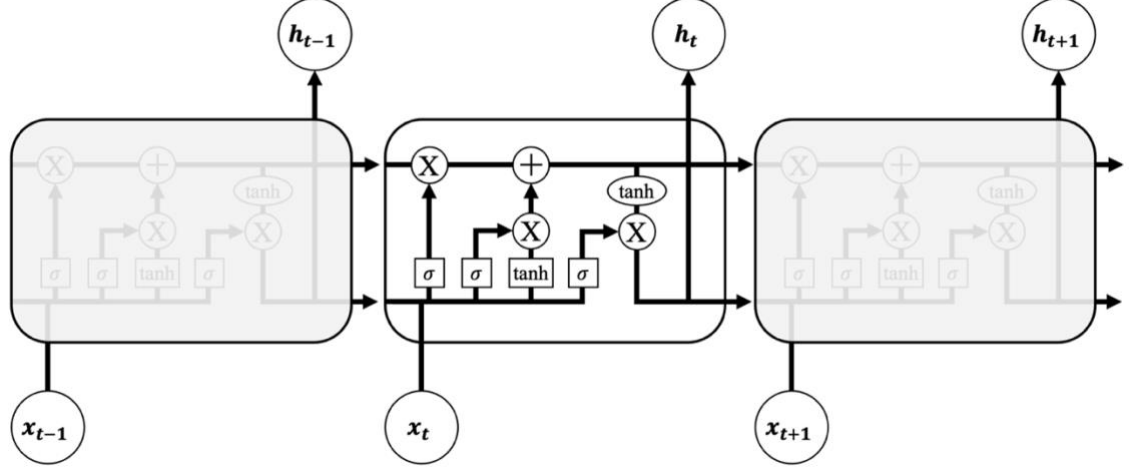


Figure 6: LSTM Structure following Olah (2015).

This part provides a step-by-step explanation of LSTM based on Olah (2015) and Vishwas & Patel (2020): When an input X_t is considered, the first step is to decide what information the model should forget. This decision is made in a sigmoid layer called forget gate layer

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f). \quad (3.46)$$

The next step is to decide what information the model should store and is divided into two parts. First, a sigmoid layer called input gate layer

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (3.47)$$

decides which values to update. Second, a tanh layer creates a vector of candidate values

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C). \quad (3.48)$$

To calculate the new cell state C_t , the previous cell state C_{t-1} is first multiplied by f_t (3.46), i.e., it forgets irrelevant information. Next, the product of i_t (3.47) and $\tilde{C}_t$ (3.48) is added to C_t , i.e., the new information is added. Thus, the new cell state can be written as

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t. \quad (3.49)$$

The final step is to determine the output. At this stage, the LSTM decides which parts of the cell state should be exposed as the hidden state. This process has two parts. First, a sigmoid layer called output gate

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (3.50)$$

controls which components of the cell state are relevant for the output. Second, the cell state is passed through a tanh function to ensure its values remain within a bounded range. The resulting hidden state is then

$$h_t = o_t * \tanh(C_t), \quad (3.51)$$

which represents a filter version of the cell state. This hidden state is not only the output at t but also serves as an input at h_{t-1} in the forget gate layer (3.46) in the next time step.

4 Methodology

This thesis examines and compares the forecasting accuracy of traditional economic time series models and DL approaches using historical data from the S&P 500. The underlying methodology is divided into six parts: (1) data collection and description, (2) data cleaning and preparation, (3) exploratory data analysis, (4) model evaluation metrics, (5) time series cross-validation, and (6) model implementation.

4.1 Data Collection & Description

To develop and compare the different models, this thesis uses historical data from the S&P 500 since it constitutes a popular index for comparative studies focusing on forecasting performances (Sezer et al., 2020). The S&P 500 is commonly regarded as the foremost indicator of large-cap U.S. equities, includes 500 leading companies, and represents roughly 80% of the market’s total capitalisation (S&P Global, 2025). The chosen dataset spans a 30-year period from January 1995 through December 2024 and was obtained from Yahoo Finance. It comprises six variables (*Table 2*) with a total of 7,552 observations. *Table 2* and *Table 3* provide descriptions of the variables and an overview of the dataset’s structure, respectively.

Variable	Description
Date	Date of the observed trading day.
Open	S&P 500 level at the beginning of the trading day.
High	Highest S&P 500 level during the trading day.
Low	Lowest S&P 500 level during the trading day.
Close	S&P 500 level at the end of the trading day.
Volume	Aggregate number of shares traded across all S&P 500 constituent companies during the trading day.

Table 2: Variable Descriptions.

Date	Open	High	Low	Close	Volume
1995-01-03	459.21	459.27	457.20	459.11	262450000
1995-01-04	459.13	460.72	457.56	460.71	319510000
1995-01-05	460.73	461.30	459.75	460.34	309050000
...	...	...	...	...	...
2024-12-31	5919.74	5929.74	5868.86	5881.63	3128350000

Table 3: Structure of the Dataset.

4.2 Data Cleaning and Preparation

Ensuring the completeness, reliability and suitability of the data constitutes a crucial preliminary step. A simple calculation based on an average of 252 trading days per year would suggest 7,560 observations. At first glance, this implies eight missing entries, which might raise concerns about incomplete data. However, the New York Stock Exchange was closed on nine extraordinary occasions during this period. These include four days following the September 11 attacks in 2001, two days during Hurricane Sandy in 2012, and three national days of mourning for former U.S. Presidents. After accounting for these closures, the dataset differs from the theoretical figure by only one trading day, a deviation that lies within the expected variation caused by the 252-day approximation. Accordingly, this thesis proceeds on the assumption that there are no missing dates.

Next, the daily returns are calculated (eq. 2.2). To calculate the daily volatility estimates, first a suitable proxy is selected. As outlined in chapter 2.5, Yang & Zhang's (2000) approach is the most accurate among the range-based volatility estimators. However, because its only usable for estimating the average volatility over multiple days (Yang & Zhang, 2000; Molnár, 2012), it does not comprise a suitable volatility proxy for this comparative study. Hence, this thesis reverts to using the volatility estimator of Rogers & Satchell (1991) as a volatility proxy. Accordingly, the daily volatility estimates are then calculated (eq. 2.13).

To ensure conciseness, the final datasets used for forecasting the returns and volatility, respectively, only include the relevant variables. Onwards from here, this thesis simply uses the names "return dataset" and "volatility dataset" for simplicity and conciseness (unless stated otherwise). Neither "Volume" nor, after the volatility estimates have been calculated, "Open", "High", and "Low" comprise relevant variables, and are subsequently dropped from both datasets. Further, the return dataset does not contain the variable "var" and only comprises data from January 2015 through December 2024. Thus, the final datasets only have two ("Date", "ret") and four variables ("Date", "ret", "ret_sq", and "var"), respectively. Because no return can be calculated for the first date due to the absence of the previous day's closing price, the final volatility dataset only comprises 7,551 observations. Since the return dataset is created by filtering the volatility dataset by date, no dates must be dropped, resulting in 2,516 observations. Table 4 and Table 5 display the structures of both datasets, respectively.

Date	Return (e+02)
2015-01-02	-0.03400215
2015-01-05	-1.844721
...	...
2024-12-31	-0.429401

Table 4: Structure of the Final Return Dataset.

Date	Return (e+03)	Return ² (e+05)	Variance (e+05)
1995-01-04	3.47898	1.210315	2.357580
1995-01-05	-0.80342	0.06454842	0.5306437
...	...	...	...
2024-12-31	-4.29401	1.843849	3.251228

Table 5: Structure of the Final Volatility Dataset.

To check the return series' stationarity, the Augmented Dickey-Fuller (ADF) test is conducted. It results in an ADF-statistic of -13.52 and a p-value < 0.01 for the return dataset. For the volatility dataset, it results in an ADF-statistic of -19.686 and a p-value < 0.01 , confirming the stationarity of the return series in both datasets (Table 6).

Dataset	ADF-statistic	p-value	Result
Return Dataset	-13.52	$p < 0.01$	Stationary
Volatility Dataset	-19.686	$p < 0.01$	Stationary

Table 6: ADF-statistics of Return Series in Return and Volatility Dataset.

After ensuring that the datasets have no missing values, computing the daily returns and the volatility estimates, selecting the relevant variables, and checking the return series for stationarity, this thesis confirms the suitability of both datasets for this comparative study.

4.3 Exploratory Data Analysis (EDA)

The exploratory data analysis (EDA) aims to provide a comprehensive understanding and insights regarding the datasets that could inform further analysis. To maintain clarity, this section is divided into two parts: (1) the return dataset and (2) the volatility dataset.

4.3.1 EDA Return Dataset

During the observed period, the S&P 500 has grown by nearly 186% with an annualised return of about 11.07% (Figure 7). The data shows that 1,351 trading days had a positive

return (accounting for about 53.70% of all trading days). Further 1,164 trading days (46.26%) had a negative return, while on one trading day (0.04%), no difference to the previous closing price was observed.



Figure 7: S&P 500 Closing Price (Jan 2015 to Dec 2024).

The highest and lowest returns were observed on March 24th, 2020 (8.97%) and March 16th, 2020 (-12.77%), respectively. Most returns were close to zero with a median of 0.06% and a mean of 0.04%, while 50% of the returns fell between -0.37% and 0.57%. Moreover, the distribution of the daily returns is slightly negatively skewed and has a leptokurtic distribution with kurtosis greater than 3, displaying heavy tails. Table 7 shows the descriptive statistics of the daily closing returns.

Min	25%	Median	Mean	75%	Max	Skewness	Kurtosis
-12.77%	-0.37%	0.06%	0.04%	0.57%	8.96%	-0.81	15.73

Table 7: Descriptive Statistics of the Returns (Jan 2015 to Dec 2024)

Figure 8 displays the returns throughout the observed period. The return series clearly highlights the existence of volatility clusters, e.g., in the beginning of the COVID-19 pandemic in 2020. Moreover, Figure 8 highlights the stationarity of the return series, allowing to visually confirm the result of the previously conducted ADF test.

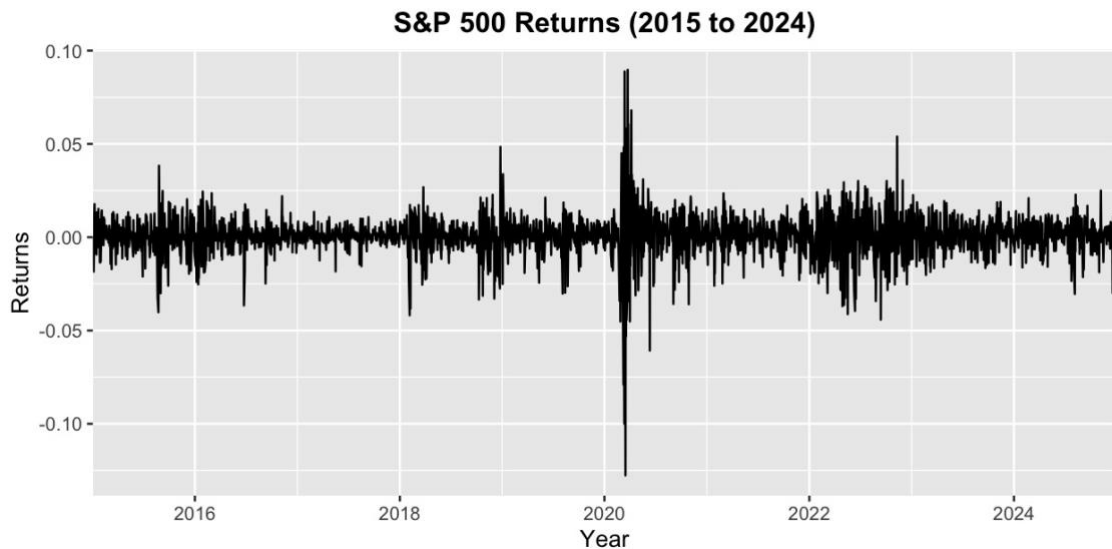


Figure 8: Return Series of S&P 500 (Jan 2015 to Dec 2024).

4.3.2 EDA Volatility Dataset

During the observed period, the S&P 500 has grown by nearly 1,181% with an annualised return of about 8.87% (Figure 9). The data shows that 4,072 trading days had a positive return (accounting for about 53.93% of all trading days). Further 3,475 trading days (46.02%) had a negative return, while on four trading days (0.05%), no difference to the previous closing price was observed.



Figure 9: S&P 500 Closing Price (Jan 1995 to Dec 2024).

The highest and lowest returns were observed on October 13th, 2008 (10.95%) and March 16th, 2020 (-12.77%), respectively. Most returns were close to zero with a median of 0.07% and a mean of 0.03%, while 50% of the returns fell between -0.47% and 0.60%.

Moreover, the distribution of the daily returns is slightly negatively skewed and has a leptokurtic distribution with kurtosis greater than 3, displaying heavy tails. Table 8 shows the descriptive statistics of the daily closing returns.

Min	25%	Median	Mean	75%	Max	Skewness	Kurtosis
-12.77%	-0.47%	0.07%	0.03%	0.60%	10.96%	-0.41	13.21

Table 8: Descriptive Statistics of the Returns (Jan 1995 to Dec 2024).

Figure 10 displays the returns throughout the observed period. The return series clearly highlights the existence of volatility clusters, e.g., during the 2008 financial crisis or in the beginning of the COVID-19 pandemic in 2020. Moreover, Figure 10 highlights the stationarity of the return series, allowing to visually confirm the result of the previously conducted ADF test.

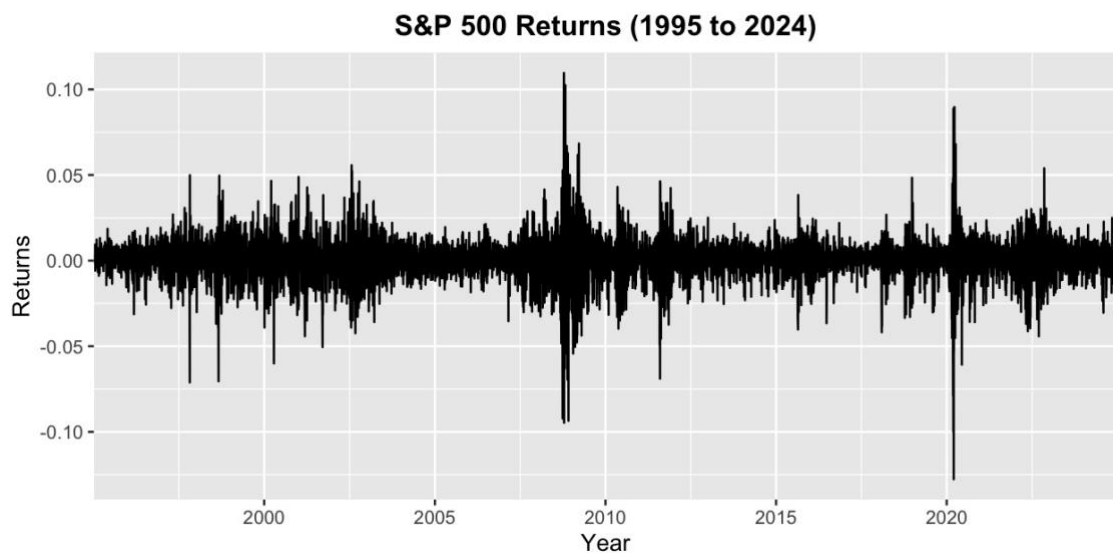


Figure 10: Return Series of S&P 500 (Jan 1995 to Dec 2024).

The highest volatility estimate was observed on October 10th, 2008 (8.79%). Looking at the lowest volatility estimate highlights how the Rogers-Satchell estimator can return values of zero. The lowest non-zero volatility estimate was observed on August 16th, 2016 (0.01%). The volatility estimates have a median of 0.57% and a mean of 0.70%, while 50% of the values fell between 0.36% and 0.89%. Moreover, the distribution of the volatility estimates is slightly positively skewed and has a leptokurtic distribution with kurtosis greater than 3, displaying heavy tails. Table 9 shows the descriptive statistics of the daily volatility estimates.

Min	25%	Median	Mean	75%	Max	Skewness	Kurtosis
0%	0.36%	0.57%	0.70%	0.89%	8.79%	3.2514	21.1393

Table 9: Descriptive Statistics of the Daily Volatility Estimates (Jan 1995 to Dec 2024)

4.4 Model Evaluation Metrics

To adequately evaluate the models' performances, choosing apt evaluation metrics is essential. Since most of the actual returns are close to zero (see Table 7), this thesis refrains from using percentage errors like MAPE, as they would return extreme values for most time steps (Hyndman & Anthanasopoulos, 2014). Moreover, they would even become infinite or undefined, as one trading day had an actual return of zero. Nonetheless, as the models forecast on the same dataset and scales, scale-dependent metrics like MSE and MAE can be used to compare the models. Thus, this thesis evaluates the one-step ahead return forecasts by using MSE and MAE. Following Patton (2011) and Bollerslev et al. (2016), this thesis then uses MSE and QLIKE to assess the one-step ahead volatility forecasts.

4.5 Time Series Cross-Validation

This thesis uses time series cross-validation as a more sophisticated version of simply splitting the data into training and test data, where the forecasting accuracy is computed by averaging over the test sets (Hyndman & Anthanasopoulos, 2021). For both parts, the cross-validation approach consists of twelve folds, as displayed in Figure 11 and Figure 12.



Figure 11: Train/Test Split Return Dataset.

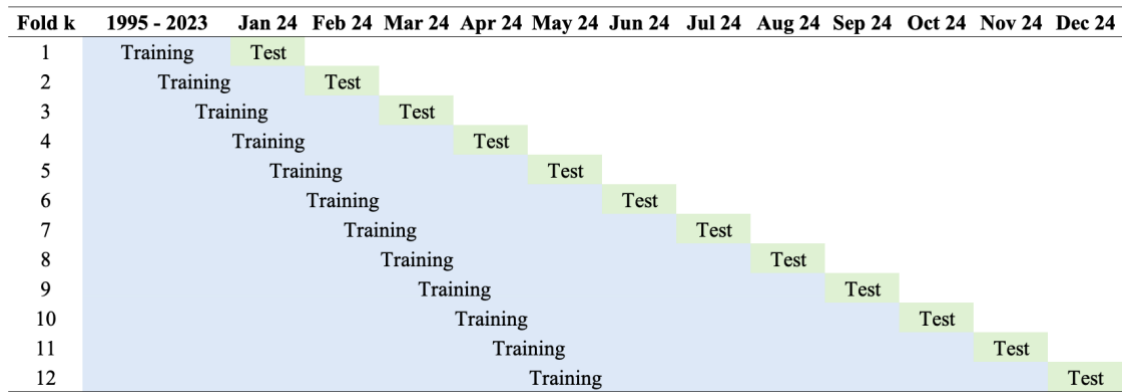


Figure 12: Train/Test Split Volatility Dataset.

In the first fold, the training data contains all observation from the respective starting points (January 1995 and January 2015) through December 2023, while the test data consists of the observations from January 2024. Each ensuing fold additionally includes the previous fold's test data. The models are retrained for each fold and must predict the one-step ahead returns and volatility, respectively. Finally, their performances are averaged over the twelve folds to obtain the overall model performance. In the return dataset, the training data contains between 2,264 and 2,495 observations, while the test contains between 19 and 23 observations (Figure 13).

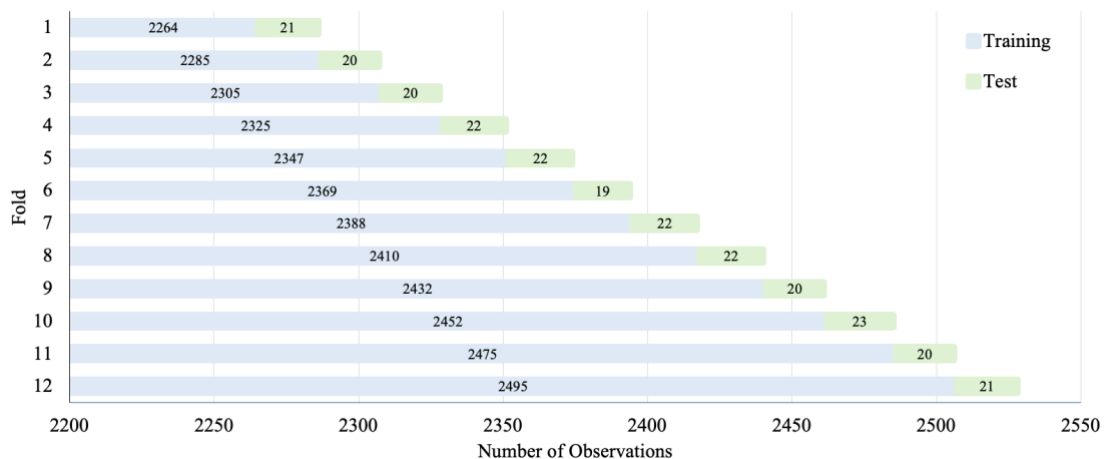


Figure 13: Lengths Training/Test Data (Return Dataset).

In the volatility dataset, the training data contains between 7,299 and 7,530 observations, while the test data contains between 19 and 23 observations (Figure 14).

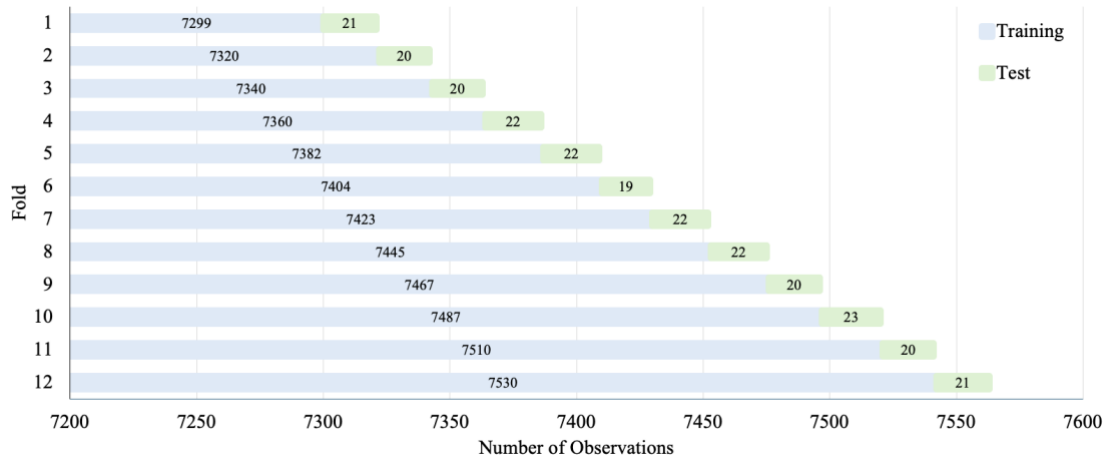


Figure 14: Lengths Training/Test Data (Volatility Dataset).

4.6 Model Implementation

The implementation of all models is conducted in *R*. While this subchapter touches upon the specifics of each model's implementation, it does not provide any code in the text. Instead, this subchapter focuses on the employed packages, design decisions, parameter estimation, and the forecasting process. The entire code used in this thesis is provided separately and, when combined with this subchapter, allows anyone to reproduce this thesis' findings.

4.6.1 ARIMA

To implement the ARIMA in *R* to forecast one-step ahead returns, this thesis primarily uses the *forecast* package (Hyndman et al., 2025). Schematically, this thesis follows Hyndman & Athanasopoulos' (2014) general process (Figure 15) for the model selection, validation, and forecasting.

Instead of manually determining the optimal model order, this thesis uses the automated algorithm *auto.arima()*, which efficiently searches through numerous combinations of p , d , and q and selects the best model by minimising the Akaike Information Criterion (AIC). Since the return series has already been differenced from the original closing prices (eq. 2.2) and confirmed to be stationary (see chapter 4.2), the order of the integration (d) is zero, effectively reducing the search to the best ARMA(p, q) model. After the best model has been selected for a given training fold, its residuals are checked to assess its suitability. This is achieved by conducting a Portmanteau test (Ljung-Box test). As all models provide non-significant p -values (Table 10), confirming the absence of any autocorrelations, they are adequate for calculating the forecasts.

Fold	Model	p-value
1	(2,0,2)	0.2172014
2	(2,0,2)	0.2048179
3	(2,0,2)	0.1984154
4	(2,0,2)	0.1908559
5	(2,0,2)	0.1882735
6	(2,0,2)	0.1744290
7	(2,0,2)	0.1732934
8	(2,0,2)	0.1772112
9	(2,0,2)	0.1476619
10	(2,0,2)	0.1689305
11	(2,0,2)	0.1649023
12	(2,0,2)	0.1481717

Table 10: ARIMA Order and Ljung-Box Test Results.

Once a model is deemed adequate, the *forecast()* function is used to generate a one-step ahead predictions for each day in the corresponding test set. The entire workflow consisting of automated selection, diagnostic checking, and forecasting is repeated for each of the twelve cross-validation folds described in chapter 4.5. The models' performances are assessed by first comparing the forecasts against the actual returns for each fold individually by calculating the MSE (eq. 2.8) and MAE (eq. 2.9). The final performance metrics are then calculated by taking a weighted average of the per-fold scores, where the weights are determined by the number of observations in each test set (see Figure 13).

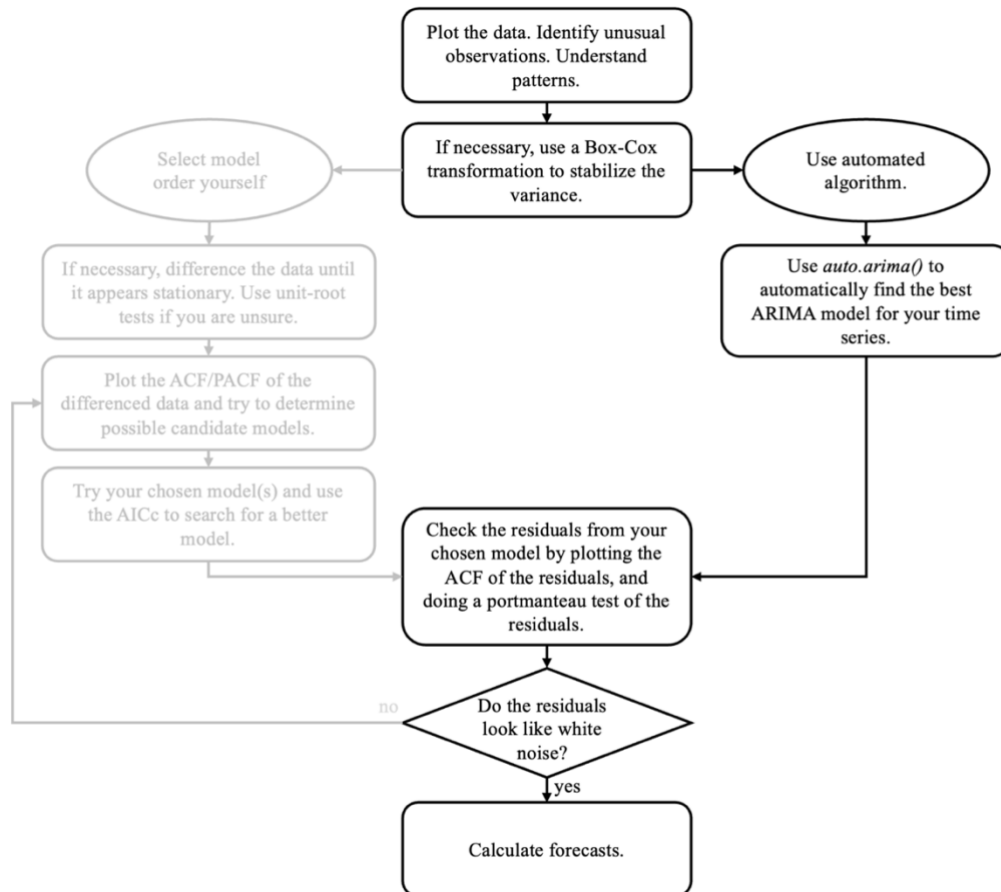


Figure 15: General Process ARIMA following Hyndman & Athanasopoulos (2014, p. 235).

4.6.2 ARCH, GARCH, and GARCH-Extensions

To implement the ARCH, as well as the GARCH and its extensions in R to forecast the one-step ahead volatility, this thesis uses the *rugarch* package (Galanos, 2025).

To determine the ARCH order, a mean model was fitted on the training data by using the *auto.arima()* function. Next, the partial autocorrelation function (PACF) of its squared residuals was analysed. Since the PACF plot (Figure 16) revealed significant spikes until lag 11, an ARCH(11) model was selected. Nonetheless, the PACF plot also motivates the use of a more parsimonious GARCH, which is likely to outperform the selected ARCH.

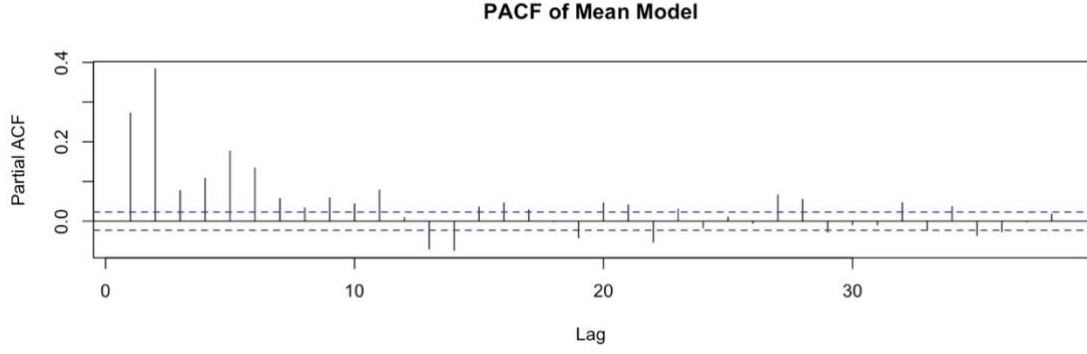


Figure 16: PACF to Determine ARCH Order.

Now, to determine the GARCH order, the orders (1,1), (1,2), (2,1), and (2,2) are fitted and their respective AICs are calculated. The fitted GARCH(2,2) model results in the lowest AIC (-6.513547). However, in accordance with the literature (e.g. Hansen & Lunde, 2005), the more parsimonious GARCH(1,1) is chosen, as it returns a very similar AIC of -6.510991. Further, the same order is chosen for all GARCH-extensions.

In each cross-validation fold, each model is specified by using the *ugarchspec()* function to define the models' core components. First, the model and its order are specified, for example, sGARCH for the standard GARCH. Next, for the mean model a simple ARMA(1,1) is chosen to ensure the volatility modelling is performed on residuals that are as close to white noise as possible. Last, a Student-t distribution is specified to account for the leptokurtosis commonly seen in financial data.

After specifying each model, it is fitted to the training data of the respective fold using the *ugarchfit()* function. Subsequently, the *ugarchforecast()* function is then used to generate the rolling one-step ahead volatility forecasts for each day of the corresponding test set. Next, the models' standard deviation forecasts are squared to obtain the final variance forecasts. In the subsequent evaluation, these variance forecasts of each fold are compared against the Rogers-Satchell estimator (eq. 2.13) by calculating the MSE (eq. 2.8) and QLIKE (eq. 2.11). The final performance metrics are then calculated by taking a weighted average of the per-fold scores, where the weights are determined by the number of observations in each test set (see Figure 14).

4.6.3 DL Models

Next, to implement the DL models in R for forecasting the one-step ahead returns and volatility, this thesis uses the *keras3* (Kalinowski et al., 2025) and *tensorflow* packages

(Allaire & Tang, 2025). To tune the hyperparameters of the DL models, this thesis uses the data from 2023 as a validation set, thus ensuring that the data from 2024 remains unseen. Subsequently, the models are trained on the data through December 2022. For both forecasting tasks, the historic mean constitutes the benchmark which each model had to beat under any circumstances. Additionally, for the volatility forecasting task, a naïve method, i.e., all forecasts are set to the previous observation's value (Hyndman & Athanasopoulos, 2014), is employed as a second benchmark as it may capture volatility clusters. The hyperparameter tuning focuses primarily on the learning rate and network size, as they constitute the two most important hyperparameters (Greff et al., 2016). Generally, the hyperparameters are estimated by experimenting with the models and minimising the target metric (MSE and QLIKE, respectively). The results are displayed in Table 11 and Table 12.

Model	MSE (e-05)	MAE (e-03)
Mean (benchmark)	6.795782	6.445857
FFNN	6.45944	5.970683
RNN	6.391988	5.919852
LSTM	6.315778	5.911445

Table 11: Model Validation (Return Forecasts).

Model	MSE (e-09)	QLIKE
Mean (benchmark)	13.16699	1.002365
Naïve (benchmark)	8.807042	579.368662
FFNN	3.71046	0.5686424
RNN	2.902613	0.5334597
LSTM	3.517706	0.5205717

Table 12: Model Validation (Volatility Forecasts).

For the return forecasting task, all three models (FFNN, RNN, and LSTM) are trained using the Adam optimiser for a maximum of 100 epochs, with the batch size set to 64. A 20-day lookback period is used for the input features. To prevent overfitting, an early stopping callback monitors the validation loss, which is calculated on a 20% split of the training data. The FFNN and RNN use the ReLU activation function (eq. 3.37), while the LSTM uses the standard tanh function (see Figure 6). Moreover, the FFNN has three hidden layers (64, 32, and 16 units), dropouts of 0.5 applied after each one, and a learning rate of 0.0005. Next, the RNN has two hidden layers (64 and 32 units), dropouts of 0.5,

and a learning rate of 0.0001. Last, the LSTM has two hidden layer (32 and 8 units), dropouts of 0.8, and a learning rate of 0.001 (see Table 13).

Parameter	FFNN	RNN	LSTM
Lookback	20	20	20
Layers	4 (incl. output)	3 (incl. output)	3 (incl. output)
Units	64 - 32 - 16 - 1	64 - 32 - 1	32 - 8 - 1
Dropout	0.5 - 0.5 - 0.5	0.5 - 0.5	0.8 - 0.8
Activation	ReLU	ReLU	tanh
Optimiser	Adam	Adam	Adam
Learning rate	0.0005	0.0001	0.001

Table 13: DL Hyperparameters (Return Forecasts).

Next, for the volatility forecasting task, most settings remain the same compared to the previous task, with only the batch size being increased from 64 to 128. However, the model architectures changed. For instance, the FFNN has four hidden layers (64, 64, 32, and 16 units), dropouts of 0.5 applied after the second and third layer, and a learning rate of 0.0001. Next, the RNN has four hidden layers (64, 64, 64, and 8 units), dropouts of 0.3, 0.2, and 0.2 applied after the first three layers, and a learning rate of 0.0001. Last, the LSTM has four hidden layers (64, 64, 32, and 8 units), dropouts of 0.2 applied after the first three layers, and a learning rate of 0.0005 (see Table 14).

Parameter	FFNN	RNN	LSTM
Lookback	20	20	20
Layers	5 (incl. output)	5 (incl. output)	5 (incl. output)
Units	64 - 64 - 32 - 16 - 1	64 - 64 - 64 - 8 - 1	64 - 64 - 32 - 8 - 1
Dropout	0 - 0.5 - 0.5 - 0	0.3 - 0.2 - 0.2 - 0	0.2 - 0.2 - 0.2 - 0
Activation	ReLU	ReLU	tanh
Optimiser	Adam	Adam	Adam
Learning rate	0.0001	0.0001	0.0005

Table 14: DL Hyperparameters (Volatility Forecasts).

For the return forecasting task, the variable “ret” (the returns) constitutes the target variable, as the DL models ought to learn the relationship between the past returns and the one-step ahead return. Likewise, the DL models ought to learn the relationship between the past returns and the one-step ahead volatility. However, choosing an apt

target variable for the volatility forecasting task is not as straightforward due to volatility being unobservable (see chapter 2.1). Now, the main challenge lies in choosing between the variables “ret_sq” (the squared returns) and “var” (the Rogers-Satchell estimator). As outlined in chapter 2.5, the squared returns constitute a noisy volatility proxy. Further, since the evaluation metrics are calculated using the Rogers-Satchell estimator, not choosing it as the target variable may put the DL models at a disadvantage. However, since its calculation is based on the variables “High”, “Low”, “Open”, and “Close” of each day (eq. 2.13), the DL models would get richer input data compared to the econometric time series models, putting the latter at a disadvantage. On one hand, choosing the squared returns as the target variable ensures a fair comparison between both model classes when provided the same input data, since they are calculated by only using the variable “Close” (eq. 2.2). On the other hand, choosing the Rogers-Satchell estimator as the target variable allows for comparing both model classes regarding their general abilities while also refraining from using a notoriously noisy volatility proxy. Due to this thesis focusing on the model comparison, the variable “ret_sq” is chosen as the target variable to provide an even playing field.

Nonetheless, the alternative approach of using the Rogers-Satchell estimator as the target variable is treated as an extension in this study. While the R scripts only require adjusting the target variable, this thesis chooses to reuse the hyperparameters for the extension part to save computational power and time. The extension part’s results are presented separately in the chapter 5.3.

4.6.4 Hybrid Models

Prior to implementing a hybrid between an econometric model and a DL approach, this thesis first considers whether a more complex hybrid, for example, a GARCH-LSTM, could realistically offer further improvements over the econometric models.

Checking the existence of any autocorrelations in the econometric models’ squared standardised residuals constitutes the first step, since the primary motivation behind hybrid models lies in the presence of patterns which the econometric models failed to capture. Similar to checking the suitability of the ARIMA’s residuals, this is achieved by conducting a Ljung-Box test regarding the squared standardised residuals of the ARCH, GARCH, EGARCH, and TGARCH. For all four models, the test is conducted using the residuals of the initial training data (January 1995 through December 2023). While the

ARCH is providing a p-value < 0.05 , thus failing to confirm the absence of remaining ARCH effects, the other three provide non-significant p-values (Table 15). The absence of any autocorrelations in the residuals of the GARCH-type models highlights their value, as they effectively tackle the ARCH's shortcomings. Further, no statistical evidence of any remaining ARCH effects or predictable structures exist in the residuals of the tested models, as they resemble white noise series. Hence, implementing a hybrid model is not justified, as no remaining predictable structures exist, meaning no further improvements are expectable by hybrid models featuring the tested GARCH-type models.

Model	p-value	Result
ARCH (11)	0.008224	Remaining ARCH effects
GARCH(1,1)	0.1849	No remaining ARCH effects
EGARCH(1,1)	0.6569	No remaining ARCH effects
TGARCH(1,1)	0.2620	No remaining ARCH effects

Table 15: Ljung-Box Test Results for ARCH/GARCH-type models.

5 Results

This chapter now presents the forecasting performances of the models in question. First, it focuses on the one-step ahead returns, before shifting its focus to the one-step ahead volatility. Each subchapter opens by briefly stating the forecasting tasks, the compared models, and the evaluation metrics used.

5.1 Forecasting Returns

Throughout the first part, ARIMA represents the econometric time series models, while the DL approaches are represented by a FFNN, RNN, and LSTM. To improve readability, the MSE and MAE are scaled by 10^5 and 10^3 , respectively, and rounded to the second decimal when stated in the text. However, the exact numbers are provided in the tables. Additionally, the historic mean of the initial training data (January 2015 through December 2023) is calculated as a benchmark and provides context on the overall model performances.

Among all the tested models, the LSTM achieves the lowest MSE (6.36), closely followed by the RNN (MSE: 6.39) and FFNN (MSE: 6.45), which return 0.54% and 1.60% higher error rates than the LSTM, respectively. The ARIMA places last among the tested models (MSE: 6.73), as the LSTM achieves a 5.54% lower MSE. Moreover, aside from the LSTM, all models fail to outscore the historic mean (MSE: 6.36), as its MSE comes within 0.08% of the best-performing model. Likewise, the LSTM outscores all other models on the MAE (5.91), as it is again followed by the RNN (MAE: 5.92), FFNN (MAE: 5.97), and ARIMA (MAE: 6.16), whose error rates are 0.19%, 1.05%, and 4.2% higher, respectively. However, on the MAE, all models fail to outscore the historic mean (MAE: 5.90), as its MAE is 0.07% lower than the best-performing model. Table 16 displays the evaluation metrics of all four models and the historic mean.

Model	MSE (e-05)	MAE (e-03)
Mean (benchmark)	6.362426	5.904196
ARIMA(2,0,2)	6.730544	6.15742
FFNN	6.45944	5.970683
RNN	6.391988	5.919852
LSTM	6.357494	5.908426

Table 16: Model Evaluation (Return Forecasts).

Analysing the models' MSEs across the twelve folds (Table 17) highlights how they tend to behave similarly, excelling and struggling in the same folds. For instance, all models achieve their lowest MSE in fold 6 (June 2024), while returning their highest MSE in fold 8 (August 2024). The ARIMA ranks first in one fold and last in four. While the FFNN ranks first in three folds, the RNN and LSTM each do so in in four. Moreover, the FFNN and RNN also both rank last in four folds, respectively. Calculating the standard deviation (SD) for each model indicates the LSTM to be delivering the most consistent performance (SD: 3.18). The ARIMA (SD: 3.34) and RNN (SD: 3.48) rank second and third, with the FFNN (SD: 4.38) having a far higher SD than the other models. However, excluding the unusually higher MSE in fold 8 reduces these differences between the models' SDs significantly, as all four models' SDs fall between 1.90 (RNN) and 2.08 (ARIMA).

Fold	ARIMA	FFNN	RNN	LSTM
1	4.936300	3.851571	4.750185	4.474325
2	6.197794	6.854677	7.102654	6.321736
3	4.085426	5.921283	3.634843	3.809161
4	7.730768	7.439726	7.356195	7.206428
5	4.137430	3.805065	4.170056	3.597246
6	1.808939	1.654537	2.329578	1.913599
7	8.518783	7.624470	8.024139	7.685201
8	15.19214	19.54248	16.00335	14.41984
9	6.809751	7.505225	6.241421	7.333629
10	5.308855	6.536432	4.727828	4.782096
11	6.526367	5.957623	6.728919	5.913416
12	8.611133	8.268689	7.911233	7.985426

Table 17: MSE (e-05) Across All Twelve Folds (Return Forecasts).

Next, analysing the four models' MAE across the twelve folds (Table 18) paints a similar picture, as all models achieve their lowest and highest MAE in fold 6 (June 2024) and fold 8 (August 2024), respectively. The ARIMA ranks last in four folds. The FFNN, RNN, and LSTM rank first in three, five, and four folds, while also ranking last in two, four, and one, respectively. Calculating the models' SDs indicates the RNN to be most consistent (SD: 1.46), followed by the LSTM (SD: 1.52), ARIMA (SD: 1.62), and FFNN (SD: 1.91).

Fold	ARIMA	FFNN	RNN	LSTM
1	4.985834	4.656282	5.304424	5.120765
2	6.133974	6.321128	6.761373	6.019708
3	5.478687	6.477862	4.966357	5.234282
4	7.361374	7.193481	6.913446	7.126998
5	5.089015	4.985123	5.277234	4.786031
6	3.378382	2.826265	3.718406	3.103410
7	7.557868	6.997282	6.735448	6.887528
8	9.884023	10.951107	9.688817	9.408598
9	5.799180	5.726703	5.916489	6.032777
10	6.647851	6.063835	5.257847	5.265233
11	5.742179	5.846322	5.932951	5.632722
12	6.647851	6.362214	5.908215	6.172339

Table 18: MAE (e-03) Across All Twelve Folds (Return Forecasts).

To briefly summarise, all four models show similar tendencies, as they struggle and excel in the same folds. Further, the ARIMA consistently performs worse than the DL approaches. Here, the LSTM tends to provide more consistent performances than the RNN and FFNN. Nonetheless, both the econometric model and the DL approaches struggle to beat the historic mean, which outperforms all models on the MAE and all but the LSTM on MSE. The plots for each model across the twelve folds is in Appendix A. Chapter 6.1 discusses these results providing further interpretations within the context of financial time series analysis.

5.2 Forecasting Volatility

Now, in the second part, the econometric time series models' representation consists of four models. In addition to the standard ARCH and GARCH, it includes the EGARCH and TGARCH. The DL approaches are again represented by a FFNN, RNN, and LSTM. Their forecasting performances are measured over twelve folds by calculating the MSE and QLIKE. To improve the readability, the MSE is scaled by 10^9 and both metrics are rounded to the second decimal, when stated in the text. The tables continue to provide the exact numbers. In addition to the compared models, the historic mean of the initial training data (January 1995 through December 2023) and a naïve forecasting method are calculated as benchmarks and provide context on the overall model performances.

All tested models comfortably beat both benchmarks on MSE and QLIKE. The TGARCH achieves the lowest MSE (3.22) and QLIKE (0.50). Among the econometric models, the ARCH has the highest MSE (4.67) and QLIKE (0.59). In comparison, the standard GARCH's MSE (3.79) and QLIKE (0.58) are 18.84% and 2.91% lower. GARCH-extensions outperform the standard GARCH on both metrics. For instance, the EGARCH's MSE (3.34) and QLIKE (0.51) are 11.72% and 10.97% lower, while the TGARCH outperforms it by 14.96% and 13.79%, respectively. Regarding the DL approaches, the RNN achieves the lowest MSE (3.43), outperforming the FFNN (3.56) and LSTM (3.83) by 3.67% and 10.40%, respectively. Whereas the RNN (0.58) also achieves a 2.87% lower QLIKE than the FFNN (0.60), it lies 5.85% above the LSTM (0.55). Table 19 displays the evaluation metrics of all seven models and the two benchmarks.

Model	MSE (e-09)	QLIKE
Mean (benchmark)	13.82920	1.114219
Naïve (benchmark)	13.85914	2513.007282
ARCH(11)	4.666989	0.5933434
GARCH(1,1)	3.787524	0.5761025
EGARCH(1,1)	3.343325	0.5128981
TGARCH(1,1)	3.220780	0.4966439
FFNN	3.56156	0.5993031
RNN	3.430942	0.582126
LSTM	3.829379	0.5499452

Table 19: Model Evaluation (Volatility Forecasts).

Analysing the models' MSE across the twelve folds (Table 20) showcases similar albeit not the same patterns as before. All models achieve their highest MSE in fold 8 (August 2024), however only the econometric models and the RNN achieve their lowest MSE in fold 6 (June 2024). Whereas the LSTM only achieves a lower MSE in two folds, the FFNN does so in six, as its MSE in fold 6 is more than twice as high as the one of the second-worst model in the respective fold (ARCH). While the same can be observed in fold 5, the FFNN then outperforms the next best model by 38.18% in fold 11 and fares far better in the notoriously difficult to predict fold 8. Calculating the SDs indicates that the RNN is the most consistent (SD: 2.72) and far better than, for example, the LSTM

(5.13). However, the SDs are heavily affected by fold 8 and only range between 2.05 (LSTM) and 2.68 (FFNN), when it is excluded.

Fold	ARCH	GARCH	EGARCH	TGARCH
1	0.9579245	0.7718051	0.4863017	0.5001752
2	3.481064	2.544973	1.0520430	0.9367025
3	1.355464	1.307330	0.8777719	0.8875402
4	1.916651	1.281305	3.0974350	3.4491750
5	3.776205	3.433016	2.8527970	2.6160410
6	0.7565359	0.7131698	0.6350396	0.6264916
7	2.572249	1.359423	1.7109250	1.7579440
8	21.73303	16.45898	13.8729000	13.4632400
9	8.700976	9.115488	8.8235030	8.4861000
10	1.016739	1.034807	1.0585710	0.9815064
11	5.251120	3.656180	1.7853850	1.5731630
12	3.927342	3.461804	3.3985420	2.8897050
Fold	FFNN	RNN	LSTM	
1	1.381836	2.159699	1.945694	
2	1.655538	2.660577	0.5700915	
3	0.9374897	4.995292	0.5236678	
4	0.9435207	1.081694	3.024326	
5	8.198931	2.794961	2.583783	
6	1.677456	0.5991702	0.6491994	
7	2.899521	3.277764	2.144369	
8	11.27459	8.852059	18.82536	
9	7.915852	8.539795	7.657854	
10	1.569661	0.9331145	1.124806	
11	0.9724656	2.370345	2.284189	
12	3.260281	2.996910	3.833820	

Table 20: MSE (e-09) Across All Twelve Folds (Volatility Forecasts).

Analysing the model's QLIKE across the twelve folds (Table 21) highlights how the econometric models' performances tend to follow a similarly pattern across the folds. For instance, all four models achieve their highest QLIKE in fold 5 (May 2024), and while

ARCH and GARCH achieve their lowest QLIKE in fold 4 (April 2024), the extensions do so in fold 1 (January 2024).

Fold	ARCH	GARCH	EGARCH	TGARCH
1	0.3709553	0.3333761	0.2325708	0.2342141
2	0.7905508	0.7170441	0.5180285	0.4936463
3	0.5938940	0.5909258	0.5061870	0.5066520
4	0.2913402	0.2516466	0.3399449	0.3517483
5	0.9527132	0.9482998	0.8473187	0.8065776
6	0.3135966	0.3094226	0.3047127	0.2981017
7	0.4585344	0.3752525	0.3628137	0.3715340
8	0.8357838	0.8287152	0.7253542	0.6976750
9	0.4958300	0.5284000	0.4731246	0.4410464
10	0.4518830	0.4618419	0.4729412	0.4591750
11	0.7900863	0.7488958	0.5939276	0.5593063
12	0.7680687	0.8135296	0.7530617	0.7139056
Fold	FFNN	RNN	LSTM	
1	0.4526850	0.5517214	0.5154309	
2	0.5275899	0.7197837	0.4037493	
3	0.5171532	0.9477006	0.4089772	
4	0.2236304	0.2179366	0.3532989	
5	1.3073612	0.8516671	0.8298275	
6	0.4741230	0.2711116	0.2821877	
7	0.6436075	0.5991907	0.4669402	
8	0.7138294	0.5358619	0.7798173	
9	0.4117312	0.4399293	0.4362296	
10	0.5519088	0.4416961	0.4875536	
11	0.4658433	0.6551026	0.6650095	
12	0.8487610	0.7641288	0.9261945	

Table 21: QLIKE Across All Twelve Folds (Volatility Forecasts).

The DL approaches do not showcase a clear pattern. While all three models perform well in fold 4 but poorly in fold 5, big differences can be observed in multiple folds, for example, in folds 2, 3, and 5. Moreover, calculating the SD for each fold individually

confirms this, as, for instance, the DL models' average per-fold SD (0.12) is twice as high as for the econometric models (0.05). On a per-model basis, the FFNN seems most inconsistent (SD: 0.27), while the other models' SDs lie between 0.18 (TGARCH) and 0.23 (GARCH).

To briefly summarise, the econometric time series models' performances tend to show similar patterns across time. The standard ARCH and GARCH tend to perform worse than the GARCH-extensions, of which the TGARCH ranks first. Moreover, using the TGARCH as the benchmark, the DL approaches struggle to compete, as the TGARCH achieves a 6.13% lower MSE and a 9.69% lower QLIKE than the best-performing DL approach (RNN and LSTM, respectively). The plots for each model across the twelve folds is in Appendix B. Chapter 6.2 discusses these results, providing further interpretations within the context of financial time series analysis.

5.3 Extension: Forecasting Volatility

As outlined in chapter 4.6.3, using the Rogers-Satchell volatility estimator as the target variable for the DL models is viewed as an extension, as it may provide further insights into the abilities of DL models and their utility for volatility forecasting. Since the previous section highlighted the TGARCH not only as the best performing econometric model but the overall best performing model, it is also included as a benchmark for the DL models in this extension. Further, the historic mean and naïve forecasting method are again included, however they are recalculated for the Rogers-Satchell estimator. Likewise, the MSE and QLIKE remain as the evaluation metrics, with the MSE scaled by 10^9 and both metrics being rounded to the second decimal in-text.

Unsurprisingly, both the mean and naïve method perform far better than before, as they are now calculated on the same metric they are evaluated against. Purely on the MSE, the mean (4.63) even manages to outscore the ARCH (4.67), while the naïve (3.43) is only beaten by the EGARCH (3.34) and TGARCH (3.22). Nonetheless, all econometric models and DL approaches comfortably beat both benchmarks on the QLIKE. Since the DL approaches have richer input data compared to the TGARCH, the results should be interpreted cautiously. Nonetheless, the FFNN (MSE: 2.38), RNN (MSE: 1.99), and LSTM (MSE: 2.35) all achieve lower MSEs, outscoring the TGARCH by 25.99%, 38.08%, and 27.01%, respectively. Likewise, the FFNN (0.47), RNN (0.43), and LSTM (0.46) also all achieve lower QLIKE scores, outperforming the TGARCH by 5.39%,

12.47%, and 8.11%, respectively. Moreover, the RNN achieves the lowest MSE and QLIKE among the three DL approaches, with the LSTM ranking second ahead of the FFNN (Table 22).

Model	MSE (e-09)	QLIKE
Mean	4.626628	0.7562861
Naive	3.428594	0.9251762
TGARCH(1,1)	3.220780	0.4966439
FFNN	2.383766	0.4698728
RNN	1.994366	0.4347079
LSTM	2.350724	0.4563468

Table 22: Model Evaluation (Volatility Forecasts with Proxy).

Across the twelve folds (Table 23), the models' MSEs show very similar patterns. For instance, all models perform well throughout the first three folds and poorly in fold 8 and 9. Whereas the RNN tends to have the highest MSE among the DL approaches when they perform well, it also tends to have the lowest MSE when they collectively perform worse. This is further reflected in the RNN having a lower SD (2.50) than the FFNN (2.98) and LSTM (2.84) across all folds, indicating more consistent forecasts.

Fold	TGARCH	FFNN	RNN	LSTM
1	0.5001752	0.5010147	0.6116927	0.4756499
2	0.9367025	0.3759084	0.6305115	0.5331434
3	0.8875402	0.5185201	0.7115298	0.3876310
4	3.4491750	1.059193	0.9989024	1.335763
5	2.6160410	2.559862	2.247707	2.499257
6	0.6264916	0.7302506	0.7399646	0.7630598
7	1.7579440	1.846578	0.9058061	1.668997
8	13.4632400	7.944489	4.681824	8.128407
9	8.4861000	9.070550	8.974976	8.316578
10	0.9815064	0.5813453	0.4673139	0.4395982
11	1.5731630	0.6470350	0.8249500	1.304023
12	2.8897050	2.651385	2.273070	2.240191

Table 23: MSE (e-09) Across All Twelve Folds (Volatility Forecasts with Proxy).

Likewise, all models achieve their lowest QLIKE scores in fold 1 and 4, while having the worst performances in fold 5 (Table 24). As seen in Table 22, the performance margins tend to be narrower between the DL models and the TGARCH when they are evaluated by using the QLIKE. This is also reflected in the SDs, as the TGARCH (0.18) is more consistent than the FFNN (0.20), while still ranking behind the LSTM (0.17) and RNN (0.14). The plots for each model across the twelve folds is in Appendix C.

Fold	TGARCH	FFNN	RNN	LSTM
1	0.2342141	0.2598808	0.2976914	0.2487957
2	0.4936463	0.3275204	0.4188408	0.3875191
3	0.5066520	0.4051119	0.4654119	0.3420504
4	0.3517483	0.2606709	0.2553119	0.2881485
5	0.8065776	0.7673363	0.6825952	0.7365492
6	0.2981017	0.3073982	0.3261245	0.3489119
7	0.3715340	0.4838928	0.3191460	0.4490237
8	0.6976750	0.7883875	0.5018458	0.6811776
9	0.4410464	0.5285538	0.6202021	0.4318779
10	0.4591750	0.3649765	0.3126485	0.3292530
11	0.5593063	0.3720362	0.4323175	0.5151193
12	0.7139056	0.7354592	0.5941133	0.6968301

Table 24: QLIKE Across All Twelve Folds (Volatility Forecasts with Proxy).

6 Discussion

In this chapter, this thesis aims to discuss the results (chapter 5) by revisiting the previously derived hypotheses (chapter 2.6) and answering the proposed research questions. Moreover, it outlines this thesis' limitations and future research directions. Both the hypotheses and the research questions are briefly restated throughout this chapter to increase the comprehensibility of the discussion.

6.1 Model Performances when Forecasting Returns

Part one of the discussion focuses on the models' performances when forecasting the one-step ahead returns of the S&P 500 (chapter 5.1). Throughout this process, by revisiting the first two hypotheses (H1 and H2), this section aims to ultimately answer the first research question.

When forecasting the one-step ahead returns for 2024, the ARIMA performs worst among all tested models, placing last on both MSE and MAE. The top-performing LSTM achieves the lowest MSE and MAE, with the other DL architectures, FFNN and RNN, also consistently outperforming the ARIMA. In addition to the overall evaluation metrics, the ARIMA is also consistently outperformed across all individual folds. Across the twelve folds for both MSE and MAE, the ARIMA ranks last in eight out of 24 folds (or about 33%), while only ranking first once, suggesting that the DL architectures provide a performance advantage over the econometric time series model. Hence, hypothesis 1 (H1) is rejected.

H1: More complex models (FFNN, RNN, and LSTM) do not outperform econometric time series models (ARIMA) on MSE and MAE when forecasting one-step ahead daily returns for the S&P 500. → Rejected

Now, although DL architectures seem to provide more accurate one-step ahead return predictions than econometric time series models, they still fail to consistently beat the historic mean, as it achieves the second-lowest MSE (coming within 0.08% of the LSTM) and the lowest MAE among all tested models. Therefore, hypothesis 2 is confirmed.

H2: Both econometric time series models (ARIMA) and more complex models (FFNN, RNN, and LSTM) struggle to outperform the historic mean on MSE and MAE when forecasting one-step ahead daily returns for the S&P 500.

→ Confirmed

Next, after revisiting H1 and H2, this section now aims to address this thesis' first research question:

RQ1: How do deep learning architectures compare with ARIMA in forecasting one-step ahead returns of the S&P 500?

Despite DL architectures outperforming the ARIMA in forecasting the one-step ahead returns, its inability to consistently beat the historic mean suggests that deploying DL architectures in return forecasting tasks is unlikely to add significant economic value. Moreover, as demonstrated by confirming H2, predicting daily returns is exceptionally difficult, with even sophisticated models failing to provide an advantage over a simple historic mean. This indirectly affects H1, as it raises concern about the observed performance differences between the DL architectures and the ARIMA, particularly regarding its reasons. Comparing their performances across different time periods, ideally with different underlying volatility patterns, would allow to inspect whether DL architectures' observed advantage exists independently of specific market dynamics.

6.2 Model Performances when Forecasting Volatility

Next, part two of the discussion focuses on the models' performances when forecasting the one-step ahead volatility of the S&P 500 (chapter 5.2 and 5.3) by revisiting H3 through H7 and ultimately answering the second proposed research question.

The ARCH(11) selected during the model implementation achieved the highest MSE among all tested models and the second highest QLIKE, narrowly outperforming the FFNN by 0.99%. The more parsimonious standard GARCH(1,1) comfortably outperforms the more complex ARCH(11), as it achieves both a lower MSE (-18.84%) and QLIKE (-2.59%). Moreover, the GARCH achieves a lower MSE and QLIKE in ten and eleven out of twelve folds, respectively. These findings not only confirm the literature stating that even a simpler GARCH tends to dominate a more complex ARCH (Poon & Granger, 2003) but also GARCH's status as a more parsimonious and effective extension of the ARCH (Bollerslev, 1986). Therefore, H3 is confirmed.

H3: A standard GARCH(1,1) model outperforms a more complex ARCH model on MSE and QLIKE when forecasting one-step ahead daily volatility estimates for the S&P 500. → Confirmed

Nonetheless, the results also highlight how the standard GARCH is typically outperformed by its extensions (Poon & Granger, 2003). Both the EGARCH and the TGARCH outperform the standard GARCH on both the MSE and QLIKE. Further, among the three models, the GARCH achieves the highest QLIKE and MSE in eight out of twelve folds. Thus, the results confirm that GARCH-extensions incorporating the asymmetries of volatility are more adequate for volatility forecasting tasks in finance and economics. Thus, H4 is confirmed.

H4: GARCH-type models which incorporate the asymmetries of volatility (EGARCH and TGARCH) outperform a standard GARCH(1,1) model on MSE and QLIKE when forecasting one-step ahead daily volatility estimates of the S&P 500. → Confirmed

All tested models comfortably beat both benchmarks. However, closely inspecting the benchmarks themselves may provide interesting insights. First, the historic mean achieving a reasonably low albeit non-competitive QLIKE indicates that it mostly overestimates the actual volatility, as the QLIKE penalises underestimations more than overestimations (ep. 2.11), i.e., the test period (2024) is characterised by a below average volatility. Now, when calculated on the squared returns, the naïve methods' astronomically high QLIKE indicates that it tends to underestimate the volatility approximated by the Rogers-Satchell estimator. Even though its poor performance is partly due to the general differences between the volatility proxies, the squared returns tendency to underestimate the actual volatility further questions its adequateness as a realistic volatility proxy. For instance, in real-world applications, underestimating an indices' volatility might be less desirable than overestimating it, conceptually similar to leverage effects. When calculating the benchmarks on the Rogers-Satchell estimator, they perform reasonably well, which, again, is partly due to using the same proxy for their calculation and evaluation. Generally, these insights align well with the growing reservation against using squared returns as a volatility proxy (Poon & Granger, 2003). However, more importantly, all tested models comfortably beating the benchmarks' QLIKE, regardless of whether their volatility proxy, highlights (1) the predictability of volatility, (2) the economic value added by volatility forecasting models, and (3) the practicality of developing and refining volatility forecasting techniques. In the context of this thesis, it underscores the intricacies of volatility forecasting, the topic's significance, and how H5 is unequivocally confirmed.

H5: Econometric time series models and DL approaches both outscore the historic mean and the naïve method on MSE and QLIKE when forecasting one-step ahead daily volatility estimates for the S&P 500. → Confirmed

While the FFNN and RNN achieve MSEs ranking them between the standard GARCH and its extensions, their QLIKE scores rank them between the ARCH and GARCH. Their failure to outperform the econometric time series models can be attributed to FFNNs' lack of memory (Goodfellow et al., 2016; Elman, 1990) and RNNs' struggle to learn long-range dependencies (Goodfellow et al., 2016; Hochreiter & Schmidhuber, 1997). Interestingly, across the twelve folds, the FFNN and RNN do not exhibit the consistent performance patterns observed among the econometric models, as they sometimes excel where econometric models struggle, while occasionally underperforming in settings where the econometric models achieve their strongest results. However, as both the FFNN and RNN fail to outperform the econometric time series models, H6 is confirmed.

H6: Neither FFNN nor RNN outperform econometric time series models (ARCH and GARCH-type models) on MSE and QLIKE when forecasting one-step ahead daily volatility estimates for the S&P 500. → Confirmed

Despite the LSTM being designed to overcome the shortcomings of the FFNN and RNN (Hochreiter & Schmidhuber, 1997) and achieving lower QLIKE scores than both DL alternatives, as well as the ARCH and GARCH, it fails to compete with the GARCH-extensions. In fact, the top-performing TGARCH achieves a significantly lower MSE and QLIKE, outscoring the LSTM by 15.89% and 9.69%, respectively. Hence, H7 is rejected.

H7: A LSTM outperforms econometric time series models (ARCH and GARCH-type models) on MSE and QLIKE when forecasting one-step ahead daily volatility estimates for the S&P 500. → Rejected

After revisiting H3 through H7, this section now aims to address this thesis' second research question while also considering the results of the volatility forecasting task's extension, which provides further insights into the capabilities of DL models:

RQ2: How do deep learning architectures compare with GARCH-type models in forecasting the one-step ahead volatility of the S&P 500?

This thesis' findings indicate that, when tasked with forecasting the one-step ahead volatility, GARCH-type models incorporating the asymmetries of volatility tend to outperform not only the standard GARCH but more complex DL architectures, too. For instance, the top-performing TGARCH consistently outscores even the most advanced DL model, the LSTM. In fact, the other two tested DL models, the FFNN and RNN, struggle to compete with the GARCH-extensions, as their respective performances are more comparable to the standard GARCH. As outlined in chapter 4.6.3, it is unfair to compare the DL models trained to predict the Rogers-Satchell estimator to the GARCH-type models, as they receive richer input data. While the extended results nonetheless highlight the DL approaches' capability to handle richer input data and to provide more accurate volatility forecasts, it should be taken with two grains of salt: (1) this thesis does not include econometric models that handle richer input data, e.g., a GARCH-X model, and (2) the accuracy gains achieved by more complex models ought to be viewed against the backdrop of their lower interpretability.

In summary, all tested models add economic value by comfortably beating the historic mean and naïve benchmarks, with the GARCH-extensions accounting for volatility asymmetries proving to be more adequate for this specific forecasting task than more complex DL architectures.

6.3 Limitations

This thesis faces some important limitations to consider when interpreting the results, while also providing paths for future research.

First, this thesis relies on univariate data, using only the historical price series of the S&P 500 as input for all models. This approach was by-design to create an even playing field for directly comparing the econometric models and DL approaches. However, the approach curtails DL approaches' ability to learn complex, non-linear patterns from multiple data sources. For example, including exogenous variables (e.g. macroeconomic data like employment rate and GDP growth) could improve the forecasting accuracy of DL approaches. Studying increasingly complex DL approaches using multiple data sources is likely going to introduce cost efficiency and interpretability as key subjects for future research to consider.

Second, likewise, the results rely on using 2024 as the lone test period. Thus, insights could be dependent on 2024-specific market dynamics. However, using a 12-fold

validation approach and largely observing similar performance patterns across the folds suggests that no drastic ranking changes are expected when using a different test period. Comparing the models across different time periods still allows to challenge this thesis' findings and increase their robustness. There is undoubtedly value in assessing whether the DL approaches surprisingly outperforming the ARIMA is down to model specifications or 2024-specific market dynamics. Nonetheless, both model classes' unsurprising inability to beat the benchmarks curtails the contribution of future research and the potential for adding economic value.

Third, as outlined before, the unobservability of volatility constitutes a well-known issue in financial time series analysis and thus necessitates a proxy for the daily volatility. To avoid relying on squared returns, as Poon & Granger (2003) call for extra caution when interpreting studies using this very noisy volatility estimator, this thesis explored multiple alternatives. Choosing the Rogers-Satchell estimator not only allowed to circumvent the initial issue but also yielded less noisy estimates, thereby enabling more reliable model evaluations. Nonetheless, the chosen proxy does not account for opening jumps, leading Yang & Zhang (2000) to question the feasibility of daily volatility estimators altogether. Moreover, due its design (eq. 2.13), the Rogers-Satchell estimator returns a daily volatility of zero on trading days characterised by monotonic price movements, i.e., where $High = Open$ and $Low = Close$, or vice versa. This occurred in 136 out of 7,551 observations (1.8%). Future research could address this limitation by using high-frequency intraday data to forecast the daily volatility or employing estimators that combine multiple time intervals (e.g., Yang & Zhang, 2000) to forecast, for example, the weekly or monthly volatility.

Fourth, this thesis acknowledges the hyperparameter tuning's profound effect on the DL models' performances. Using different approaches and validation sets likely results in different hyperparameters and thus different performances. While confident in its process, this thesis does not claim to have found the perfect hyperparameter configuration. Similar to the previously outlined limitation of relying on 2024 as the lone test period, this thesis relies on 2023 as the lone validation period. Therefore, by including different time periods, future research could provide further insights, challenge this thesis' results and potentially increase their robustness. Moreover, future research could extend this thesis' scope altogether by exploring how different hyperparameter tuning strategies, for example, random search (e.g., Bergstra & Bengio, 2012), Bayesian

optimisation (e.g., Snoek et al., 2012), or sequential model-based optimisation (e.g., Hutter et al., 2011), affect DL approaches' forecasting performances and how they eventually compared against the benchmark set by specialised econometric time series models.

Fifth, a hybrid GARCH-LSTM was initially considered as part of this comparative study. However, after implementing the GARCH-type models, there were no uncaptured patterns in their residuals. Despite there not being statistical evidence of hybrid models providing further improvements in this dataset, future research could apply this methodology to other financial data or time periods where the underlying volatility dynamics might exhibit non-linearities that the standard GARCH and its extensions cannot fully capture.

7 Conclusion

This thesis compared the accuracy of econometric time series models and DL approaches when tasked with forecasting the one-step ahead returns and volatility of the S&P 500. The research confirms two central findings: (1) it is notoriously difficult for any forecasting model to consistently outperform the historic mean when forecasting daily returns and (2) specialised econometric time series models which incorporate the asymmetries of volatility tend to outperform more complex and flexible DL approaches when forecasting the volatility.

Further, interpretability is a frequently discussed subject in the realm of employing DL approaches. Therefore, in its final remark, this thesis wants to focus on the role of interpretability in the context of comparing and eventually deploying econometric models and DL approaches in economics and finance. More specifically, this thesis argues that the interpretability of forecasting models ought to be systematically included in the model selection process. For instance, by treating interpretability in the same way parsimony is treated when choosing a simpler GARCH(1,1) over a higher-order GARCH which may yield slightly better results. This pragmatic approach would ensure that model complexity is only accepted when justified by a significant and robust performance advantage.

Reference List

- Alizadeh, S., Brandt, M. W., & Diebold, F. X. (2002). Range-based estimation of stochastic volatility models. *The Journal of Finance*, 57(3), 1047-1091.
- Allaire, J., & Tang, Y. (2025). *tensorflow: R Interface to "TensorFlow"*. R package version 2.20.0.
- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *The journal of machine learning research*, 13(1), 281-305.
- Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of econometrics*, 31(3), 307-327.
- Bollerslev, T., Chou, R. Y., & Kroner, K. F. (1992). ARCH modeling in finance: A review of the theory and empirical evidence. *Journal of econometrics*, 52(1-2), 5-59.
- Bollerslev, T., Patton, A. J., & Quaedvlieg, R. (2016). Exploiting the errors: A simple approach for improved volatility forecasting. *Journal of Econometrics*, 192(1), 1-18.
- Box, G., & Jenkins, G. (1970). *Time series analysis, forecasting, and control*. Holden-Day.
- Box, G. E., Jenkins, G. M., & Reinsel, G. C. (1994). *Time series analysis: forecasting and control*. John Wiley & Sons.
- Brownlee, J. (2018). *Deep learning for time series forecasting: predict the future with MLPs, CNNs and LSTMs in Python*. Machine Learning Mastery.
- Carmona, R. (2014). *Statistical analysis of financial data in R* (Vol. 2). New York: Springer.
- Cont, R. (2001). Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative finance*, 1(2), 223-236.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4), 303-314.
- Efron, B., & Hastie, T. (2017). *Computer age statistical inference, student edition: algorithms, evidence, and data science* (Vol. 6). Cambridge University Press.

- Elman, J. L. (1990). Finding structure in time. *Cognitive science*, 14(2), 179-211.
- Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica: Journal of the econometric society*, 987-1007.
- Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European journal of operational research*, 270(2), 654-669.
- Francq, C., & Zakoian, J. M. (2010). *GARCH models: structure, statistical inference and financial applications*. John Wiley & Sons.
- Galanos, A. (2025). *rugarch*: Univariate GARCH models. R package version 1.5-4.
- Goodfellow, I., Bengio, Y., Courville, A., & Bengio, Y. (2016). *Deep learning*, 1(2). Cambridge: MIT press.
- Greff, K., Srivastava, R. K., Koutník, J., Steunebrink, B. R., & Schmidhuber, J. (2016). LSTM: A search space odyssey. *IEEE transactions on neural networks and learning systems*, 28(10), 2222-2232.
- Han, J., & Moraga, C. (1995). The influence of the sigmoid function parameters on the speed of backpropagation learning. In *International workshop on artificial neural networks* (pp. 195-201). Berlin, Heidelberg: Springer Berlin Heidelberg.
- Hansen, P. R., & Lunde, A. (2005). A forecast comparison of volatility models: does anything beat a GARCH (1, 1)? *Journal of applied econometrics*, 20(7), 873-889.
- Haykin, S. (2009). *Neural Networks and Learning Machines* (3rd ed.). Upper Saddle River, NJ: Pearson Education.
- Heaton, J. B., Polson, N. G., & Witte, J. H. (2017). Deep learning for finance: deep portfolios. *Applied Stochastic Models in Business and Industry*, 33(1), 3-12.
- Henrique, B. M., Sobreiro, V. A., & Kimura, H. (2019). Literature review: Machine learning techniques applied to financial market prediction. *Expert Systems with Applications*, 124, 226-251.

- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.
- Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5), 359-366.
- Hutter, F., Hoos, H. H., & Leyton-Brown, K. (2011, January). Sequential model-based optimization for general algorithm configuration. In *International conference on learning and intelligent optimization* (pp. 507-523). Berlin, Heidelberg: Springer Berlin Heidelberg.
- Hyndman, R. J., & Athanasopoulos, G. (2014). *Forecasting: principles and practice*. OTexts.
- Hyndman R.J., Athanasopoulos G., Bergmeir C., Caceres G., Chhay L., O'Hara-Wild M., Petropoulos F., Razbash S., Wang E., Yasmeeen F. (2025). *forecast: Forecasting functions for time series and linear models*. R package version 8.24.0.
- Hyndman, R.J., & Athanasopoulos, G. (2021) *Forecasting: principles and practice*, 3rd edition, OTexts: Melbourne, Australia.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning: With Applications in R* (2nd ed.). Springer.
- Kakade, K., Jain, I., & Mishra, A. K. (2022). Value-at-Risk forecasting: A hybrid ensemble learning GARCH-LSTM based approach. *Resources Policy*, 78, 102903.
- Kalinowski, T., Allaire, J., & Chollet, F. (2025). *keras3: R Interface to "Keras"*. R package version 1.4.0.
- Kim, H. Y., & Won, C. H. (2018). Forecasting the volatility of stock price index: A hybrid model integrating LSTM with multiple GARCH-type models. *Expert Systems with Applications*, 103, 25-37.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *nature*, 521(7553), 436-444.
- Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). The M4 Competition: Results, findings, conclusion and way forward. *International Journal of forecasting*, 34(4), 802-808.

- Molnár, P. (2012). Properties of range-based volatility estimators. *International Review of Financial Analysis*, 23, 20-29.
- Mullainathan, S., & Spiess, J. (2017). Machine learning: an applied econometric approach. *Journal of Economic Perspectives*, 31(2), 87-106.
- Nelson, D. B. (1991). Conditional heteroskedasticity in asset returns: A new approach. *Econometrica*, 59(2), 347-370.
- Olah, C. (2015). Understanding LSTM networks.
- Pai, P. F., & Lin, C. S. (2005). A hybrid ARIMA and support vector machines model in stock price forecasting. *Omega*, 33(6), 497-505.
- Parkinson, M. (1980). The extreme value method for estimating the variance of the rate of return. *Journal of business*, 61-65.
- Patton, A. J., & Sheppard, K. (2009). Evaluating volatility and correlation forecasts. In *Handbook of financial time series* (pp. 801-838). Berlin, Heidelberg: Springer Berlin Heidelberg.
- Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of econometrics*, 160(1), 246-256.
- Poon, S. H., & Granger, C. W. J. (2003). Forecasting volatility in financial markets: A review. *Journal of economic literature*, 41(2), 478-539.
- Ramos-Pérez, E., Alonso-González, P. J., & Núñez-Velázquez, J. J. (2019). Forecasting volatility with a stacked model based on a hybridized Artificial Neural Network. *Expert Systems with Applications*, 129, 1-9.
- Rogers, L. C. G., & Satchell, S. E. (1991). Estimating variance from high, low and closing prices. *The Annals of Applied Probability*, 504-512.
- S&P Global. (2025). S&P 500. S&P Dow Jones Indices. <https://www.spglobal.com/spdji/en/indices/equity/sp-500/>
- Schmidt-Hieber, J. (2020). Nonparametric regression using deep neural networks with ReLU activation function. *Ann. Statist.* 48(4). 1875-1897.

- Sezer, O. B., Gudelek, M. U., & Ozbayoglu, A. M. (2020). Financial time series forecasting with deep learning: A systematic literature review: 2005–2019. *Applied soft computing*, 90, 106181.
- Shumway, R. H., & Stoffer, D. S. (2025). *Time series analysis and its applications: With R examples* (5th ed.). Springer.
- Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical bayesian optimization of machine learning algorithms. *Advances in neural information processing systems*, 25.
- Tsay, R. S. (2006). Financial time series. In S. Kotz, N. Balakrishnan, C. B. Read, & B. Vidakovic (Eds.), *Encyclopedia of statistical sciences* (2nd ed.). John Wiley & Sons.
- Tsay, R. S. (2010). *Analysis of Financial Time Series*. John Wiley & Sons.
- Vishwas, B. V., & Patel, A. (2020). Regression extension techniques for time-series data. In *Hands-on Time Series Analysis with Python: From Basics to Bleeding Edge Techniques* (pp. 99-184). Berkeley, CA: Apress.
- Wu, C., Kao, C., & Lee, C. F. (1996). Time-series properties of financial series and implications for modeling. *Journal of Accounting, Auditing & Finance*, 11(2), 277-303.
- Yang, D., & Zhang, Q. (2000). Drift-independent volatility estimation based on high, low, open, and close prices. *The Journal of Business*, 73(3), 477-492.
- Zakoian, J. M. (1994). Threshold heteroskedastic models. *Journal of Economic Dynamics and control*, 18(5), 931-955.
- Zhang, G., Patuwo, B. E., & Hu, M. Y. (1998). Forecasting with artificial neural networks: The state of the art. *International journal of forecasting*, 14(1), 35-62.
- Zhang, G. P. (2003). Time series forecasting using a hybrid ARIMA and neural network model. *Neurocomputing*, 50, 159-175.